

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

направленность (профиль)
«Технологии разработки программного обеспечения»

Выпускная квалификационная работа

Разработка системы управления контентом в сфере маркетинга на базе Strapi и
Astro

Обучающегося 4 курса
очной формы обучения
Нахатакяна Артура Романовича

Руководитель выпускной квалификационной
работы:
кандидат педагогических наук, доцент,
доцент кафедры информационных технологий и
электронного обучения
Государев Илья Борисович

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
Глава 1. Аналитическая часть	9
1.1 Особенности маркетинговых сайтов как объекта автоматизации	9
1.2 Подходы к управлению контентом: традиционные CMS и headless CMS	12
1.3 Обоснование выбора Strapi и Astro	16
1.4 Формирование требований к системе	18
1.5 Выводы по аналитической главе	23
Глава 2. Проектная часть	25
2.1 Общая архитектура системы	25
2.2 Проектирование модели данных CMS	27
2.3 Конструктор страниц на основе Dynamic Zone	29
2.4 Реализация frontend-части на Astro	30
2.5 Мультиязычность	34
2.6 Preview и защищенный доступ к черновикам	35
2.7 SEO, Open Graph и sitemap	37
2.8 Безопасность публичных сценариев	43
2.9 Тестирование	45
2.10 Выводы по проектной главе	50
ЗАКЛЮЧЕНИЕ	52
Список сокращений и условных обозначений	55
Словарь терминов	56
Список иллюстративного материала	57
Список литературы	59
ПРИЛОЖЕНИЯ	63
Приложение А. Контекстная схема CMS-first системы	63
Приложение Б. Схема модели данных CMS	64
Приложение В. Схема жизненного цикла контента	65
Приложение Г. Схема защищенного предпросмотра	66

ВВЕДЕНИЕ

Современный корпоративный маркетинговый сайт выполняет не только представительскую, но и операционную функцию: он используется для публикации страниц под маркетинговые кампании, размещения статей и кейсов, публикации вакансий, сбора откликов и поддержки поисковой видимости компании в цифровой среде. При этом ценность такой платформы определяется не только качеством визуального интерфейса, но и скоростью обновления контента, предсказуемостью публикации и возможностью передавать редакторские задачи контент-менеджеру без постоянного участия разработчика.

Для маркетинговых сайтов типична ситуация, при которой публичная витрина развивается быстрее, чем механизмы управления содержимым. В результате часть контента и интерфейсных текстов остается зашитой во frontend-коде, публикация изменений требует ручного участия разработчика, а сценарии локализации, предварительного просмотра и управления SEO-метаданными оказываются фрагментированными. Такое состояние затрудняет поддержку сайта как контентной системы и снижает эффективность взаимодействия между разработкой и маркетинговой функцией компании.

Актуальность темы

Актуальность темы обусловлена тем, что корпоративные маркетинговые сайты требуют регулярного обновления контента, поддержки нескольких языков, управления структурой страниц, публикации материалов с учетом SEO-требований и воспроизводимого сценария вывода изменений на публичную витрину. При монолитном или частично кодо-центричном подходе к разработке сайта эти задачи решаются разрозненно: одни данные хранятся в CMS, другие остаются во frontend-приложении, а публикационный процесс зависит от ручных действий и знания внутренней структуры проекта. Такая постановка подтверждается как исследованиями роли сайта в маркетинговых коммуникациях, так и прикладными работами по требованиям к CMS в retail-конуре [1; 2].

Для рассматриваемой предметной области такое противоречие проявляется между потребностью в быстром редакторском управлении маркетинговым контентом и ограничениями архитектуры, в которой управление содержанием недостаточно отделено от слоя отображения. В этих условиях актуальной стано-

вится разработка CMS-first системы, где Strapi используется как ядро управления контентом, а Astro — как публичная витрина, ориентированная на производительный рендеринг и контролируруемую публикацию.

Дополнительную значимость для данной предметной области имеет возможность предварительной сборки публичных страниц frontend-слоя. Для маркетингового сайта, в котором значительная часть материалов представляет собой контентные страницы с относительно редким изменением между публикациями, такой подход позволяет отдавать заранее подготовленное представление страницы, уменьшать зависимость витрины от runtime-логики CMS и повышать предсказуемость публикационного контура. В работе этот эффект рассматривается как архитектурное следствие связки Strapi + Astro и схемы webhook -> rebuild, а не как абстрактное свойство любой CMS.

Практическая значимость такой постановки усиливается тем, что работа выполняется не как абстрактное исследование технологий, а как разработка проектной реализации в монорепозитории Nx + pnpm, включающем backend на Strapi 5 и frontend на Astro 6. Это позволяет рассматривать тему ВКР как инженерную задачу построения целостной модели управления страницами, статьями, проектами, вакансиями и откликами с учетом локализации, режима предварительного просмотра, SEO, схемы webhook -> rebuild и контура развертывания.

Объект и предмет исследования

Объектом исследования являются процессы управления контентом корпоративного маркетингового сайта.

Предметом исследования являются архитектурные и программные решения для построения CMS-first веб-платформы на базе Strapi и Astro, обеспечивающей редакторское управление страницами, статьями, проектами, вакансиями и откликами без непосредственного вмешательства в исходный код публичной витрины.

Цель и задачи работы

Целью работы является разработка системы управления контентом для корпоративного маркетингового сайта на базе Strapi и Astro, обеспечивающей управление страницами, статьями, кейсами и вакансиями, поддержку локалей ru и en, предварительного просмотра, SEO-настроек и воспроизводимого сценария публикации без постоянного участия разработчика.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать особенности маркетинговых сайтов, подходы к управлению контентом и обосновать выбор CMS-first архитектуры на базе Strapi и Astro;
2. сформировать функциональные и нефункциональные требования к системе, а также спроектировать ее архитектуру и модель данных для ключевых контентных сущностей;
3. реализовать CMS-first контур управления страницами и ключевым контентом, включая коллекцию pages, конструктор страниц на основе Dynamic Zone и вынос основных интерфейсных текстов в Strapi;
4. реализовать ключевые возможности публичной витрины, включая поддержку локалей ru/en, режим предварительного просмотра, управление SEO-полями, Open Graph и генерацию sitemap;
5. организовать эксплуатационный контур системы, включающий роли и права редакторов, защиту публичных сценариев, публикацию через схему webhook -> rebuild, а также deployment frontend и CMS как отдельных Docker-приложений под управлением Dokploy;
6. провести проверку ключевых пользовательских и редакторских сценариев и оценить полученный результат по функциональным, локализационным, SEO- и эксплуатационным характеристикам.

Методы исследования и разработки

В работе используются методы системного и сравнительного анализа, применяемые для исследования особенностей маркетинговых сайтов и выбора архитектурного подхода к управлению контентом. Для проектирования системы используются методы структурного анализа предметной области, моделирования контентных сущностей и их связей, а также архитектурного проектирования распределенной веб-платформы с разделением контентного и витринного слоев.

В практической части применяются методы компонентной разработки frontend-приложений, интеграционной настройки Strapi и Astro, формирования типизированного API-контракта на основе OpenAPI, конфигурирования deployment-процесса и функционального тестирования сценариев публикации, локализации и работы публичных форм. Дополнительно используются приемы

инженерной валидации, основанные на сопоставлении поставленных требований с фактическим состоянием кода, конфигурации и проверяемых сценариев системы.

Теоретическая значимость

Теоретическая значимость работы заключается в систематизации подхода к построению CMS-first архитектуры для маркетингового сайта, в которой управление контентом, локализация, публикация, поисковая оптимизация и публичный рендеринг рассматриваются как взаимосвязанные части одной инженерной системы. Работа уточняет принципы разграничения ответственности между CMS-ядром и frontend-витриной, а также показывает, каким образом контентная модель, сценарии публикации и deployment-конфигурация должны проектироваться согласованно, а не как набор изолированных доработок.

Дополнительная теоретическая ценность состоит в формализации требований к предметной области корпоративного маркетингового сайта, где ключевое значение имеют не только отображение контента, но и редакторские сценарии: создание страниц из блоков, мультиязычность, предварительный просмотр, управление метаданными и воспроизводимая публикация на публичную витрину.

Практическая значимость

Практическая значимость работы определяется тем, что ее результат ориентирован на непосредственное использование в реальном проекте. Разработанная система обеспечивает перенос ключевого контента и редакторских операций в Strapi, снижает зависимость маркетинговой команды от разработчика при типовых изменениях и обеспечивает единый путь публикации материалов на публичный сайт.

Для контент-менеджера и маркетолога практическая ценность решения состоит в возможности управлять страницами, статьями, проектами, вакансиями и связанными метаданными через специализированный административный интерфейс, а не через редактирование исходного кода. Для проекта в целом практическая значимость выражается в централизованной модели данных, поддержке локалей ru/en, подготовке сценария предварительного просмотра, организации SEO-данных и воспроизводимого контура развертывания, в котором frontend и CMS развертываются как отдельные Docker-приложения под управлением Dokploy. Разработанная в рамках ВКР система подтверждается кодом, конфигу-

рацией и воспроизводимыми тестовыми сценариями, поэтому ее практическая значимость проявляется как реализованный инженерный результат.

Дополнительно практическая ценность выбранной архитектуры связана с тем, что публичная витрина может собираться заранее на стороне frontend-слоя. Для контентных страниц это создает предпосылки для более предсказуемой отдачи результата и снижает необходимость формировать представление страницы при каждом обращении пользователя. Данный вывод в работе трактуется как архитектурное преимущество выбранной схемы, а не как заранее измеренное количественное утверждение о производительности.

Краткое описание результата работы

Кратким результатом работы является разработка архитектурного и программного решения для перехода корпоративного маркетингового сайта к CMS-first модели, в которой Strapi выступает центром управления контентом, а Astro выполняет роль публичной витрины. Целевая система охватывает управление страницами, статьями, проектами, вакансиями и откликами, сборку страниц на основе Dynamic Zone, поддержку локалей ru/en, режим предварительного просмотра, SEO, Open Graph, sitemap, роли и права, а также публикационный контур webhook -> rebuild.

В рамках работы сформирован монорепозитарий с самостоятельными приложениями Strapi и Astro, реализованными модулями статей, проектов, вакансий, откликов, маркетинговых страниц и редакторских сценариев публикации. В тексте диплома раскрываются архитектура системы, состав реализованных модулей, границы принятого объема и подтверждение итогового результата архитектурой, конфигурацией и тестированием.

Структура работы

Работа состоит из введения, двух глав, заключения, списка сокращений, словаря терминов, списка использованных источников и приложений. В первой главе рассматриваются особенности маркетинговых сайтов как объекта автоматизации, сравниваются традиционные и headless подходы к управлению контентом, обосновывается выбор Strapi и Astro, а также формируются требования к системе. Во второй главе описываются архитектура решения, модель данных, сценарии публикации, подход к локализации, deployment pipeline, вопросы безопасности и результаты тестирования. В заключении подводятся итоги работы, форму-

лируются достигнутые результаты, ограничения и направления дальнейшего развития системы.

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

В первой главе маркетинговый сайт рассматривается не как набор отдельных страниц, а как предметная область, в которой эксплуатационная ценность системы определяется скоростью, управляемостью и предсказуемостью контентных изменений. Поэтому аналитика выстроена в последовательности “особенности предметной области -> архитектурный выбор -> требования к системе”: сначала выделяются свойства маркетинговой витрины как объекта автоматизации, затем оцениваются подходы к управлению контентом и после этого формируется набор требований, который используется как основание для проектных решений второй главы. В рамках ВКР такая логика принципиальна, поскольку разрабатывается не просто публичный frontend, а CMS-first система управления контентом для корпоративной маркетинговой платформы.

1.1 ОСОБЕННОСТИ МАРКЕТИНГОВЫХ САЙТОВ КАК ОБЪЕКТА АВТОМАТИЗАЦИИ

Роль маркетингового сайта в цифровых коммуникациях компании

Маркетинговый сайт в современной организации выполняет роль публичной цифровой витрины, через которую компания представляет продукты и услуги, формирует профессиональную репутацию, публикует экспертные материалы, демонстрирует выполненные проекты и поддерживает коммуникацию с потенциальными клиентами и соискателями. В отличие от простого информационного сайта, он должен быстро адаптироваться к новым маркетинговым кампаниям, изменениям позиционирования, запуску новых направлений и потребности в регулярном обновлении содержимого [1; 2].

Однако для инженерного анализа важен не сам факт наличия таких материалов, а режим их изменения. В маркетинговом контуре содержание сайта обновляется не эпизодически, а в ответ на кампании, запуск новых предложений, публикацию экспертных материалов, изменения позиционирования и кадровые сценарии. Следовательно, система должна поддерживать частые редакторские операции как нормальный режим работы, а не как исключение. Это смещает фокус проектирования с разовой разработки шаблонов на организацию

управляемого контентного контура, в котором типовые изменения не требуют вмешательства в код публичной витрины.

Из этого вытекает и второй важный вывод: ценность маркетингового сайта определяется не только качеством визуального представления, но и тем, насколько быстро и предсказуемо он позволяет провести материал через цикл “подготовка -> проверка -> публикация”. Если типовые контентные изменения зависят от разработчика, возникает организационное узкое место: маркетинговая команда не может оперативно обновлять страницы, публикации и метаданные, а сопровождение сайта начинает смешиваться с задачами развития frontend. Следовательно, объектом автоматизации становится не только отображение контента, но и сам редакторский процесс, включая его роли, точки контроля и публикационный контур.

Для корпоративного маркетингового сайта также существенны требования к поисковой видимости, локализации и управляемой публикации. Контент должен быть пригоден для индексации, сопровождаться метаданными, поддерживать локализованные версии и публиковаться в воспроизводимом контуре, не зависящем от ручных действий в кодовой базе. Эти свойства делают сайт не просто набором маршрутов, а контентной системой со своими пользовательскими ролями, моделями данных и эксплуатационными сценариями. Для русскоязычных работ по информационной архитектуре и корпоративным сайтам также характерен акцент на устойчивой навигации, структурной согласованности контента и централизованном управлении информационным слоем [3—5].

В таблице 1.1 сведены ключевые свойства рассматриваемой предметной области и их прямые инженерные последствия для проектируемой системы.

Таблица 1.1 – Связь особенностей маркетингового сайта с инженерными требованиями к системе

Особенность предметной области	Инженерное следствие для системы
Частые изменения контента в ответ на кампании, публикации и изменения позиционирования	Типовые обновления должны выполняться через управляемые контентные сущности и редакторский интерфейс, без ручного изменения frontend-кода.
Необходимость независимого редакторского цикла	Система должна разделять роли редактирования и разработки, поддерживать черновой режим, проверку материала и публикацию как самостоятельный контур.

Продолжение таблицы 1.1

Особенность предметной области	Инженерное следствие для системы
Мультиязычность публичного контента	Локализация должна проектироваться на уровне данных, маршрутов и редакторских сценариев, а не сводиться к разрозненным текстовым вставкам в шаблонах.
Управление SEO и метаданными	Метаданные должны быть частью контентной модели, чтобы редактор мог управлять индексируемостью, заголовками, описаниями и Open Graph без участия разработчика.
Необходимость предварительного просмотра до публикации	Требуется отдельный сценарий preview, позволяющий проверить будущую публичную версию до вывода материала в основной контур.
Требование воспроизводимой схемы публикации	Публикация должна рассматриваться как формализованный процесс обновления витрины, а не как набор ручных операций в кодовой базе и инфраструктуре.

Контентные сущности и редакторские сценарии

Для рассматриваемой предметной области характерен ограниченный, но содержательно важный набор сущностей. К ним относятся произвольные маркетинговые страницы, статьи, проекты, вакансии, отклики на вакансии, а также глобальные данные сайта. Каждая из этих сущностей имеет собственный жизненный цикл и собственные требования к редактированию. Например, маркетинговая страница должна собираться из переиспользуемых блоков, статья требует структурированного текста, обложки и авторства, проект должен показывать портфолио выполненных работ, а вакансия должна не только отображаться на витрине, но и быть связана со сценарием отправки отклика.

Инженерно значимо то, что перечисленные сущности не сводятся к одному шаблону “страница с текстом”. Они требуют разных структур данных, разных правил публикации и разных связей между редакторским и публичным контуром. Если система хранит такие материалы как слабо структурированный набор полей, то редакторский процесс быстро становится зависимым от частных доработок интерфейса и ручной правки представления. Следовательно, уже на

уровне предметной области возникает требование к явной контентной модели, в которой страницы, статьи, проекты, вакансии и глобальные данные оформлены как самостоятельные сущности со своими атрибутами и жизненным циклом.

Отсюда вытекает основной редакторский сценарий: контент-менеджер или маркетолог должен создать или изменить материал, заполнить структурированные поля, при необходимости подготовить локализованную версию, проверить публичное представление и опубликовать результат без ручного изменения frontend-кода. В этой логике SEO-поля, метаданные Open Graph, локализованные маршруты и предварительный просмотр перестают быть второстепенными опциями интерфейса и становятся обязательными элементами архитектуры редакторского контура.

Таким образом, маркетинговый сайт представляет собой систему, в которой ценность создается не только конечной HTML-страницей, но и качеством управления ее содержимым. Это отличает рассматриваемую предметную область от проектов, где достаточно один раз сверстать несколько шаблонов и редко возвращаться к их содержимому. Для ВКР по разработке системы управления контентом такое различие принципиально: требуется проектировать именно управляемую контентную платформу, а не только витринный интерфейс. Именно поэтому следующим шагом анализа становится сравнение архитектурных подходов к управлению контентом: необходимо определить, какая модель CMS лучше соответствует требованиям независимого редакторского цикла, мультиязычности, управляемого SEO, preview и воспроизводимой публикации.

1.2 ПОДХОДЫ К УПРАВЛЕНИЮ КОНТЕНТОМ: ТРАДИЦИОННЫЕ CMS И HEADLESS CMS

Ограничения традиционного CMS-подхода

Традиционная CMS обычно объединяет в одном контуре хранение данных, административный интерфейс, шаблоны отображения и механизм вывода страницы пользователю. Такая архитектура хорошо подходит для типовых сайтов, в которых структура представления относительно стабильна, а публичный слой тесно связан с самой системой управления контентом. Для небольших задач это снижает порог входа и ускоряет старт разработки, поскольку многие возможности доступны из коробки [6]. Подобная логика характерна и для CMS-ориентированных framework-подходов, автоматизирующих создание типо-

вых веб-сайтов и CRUD-контуров, где приоритет отдается скорости запуска и унификации административной части [7].

Для задачи маркетингового сайта принципиально, что преимущества такого подхода относятся прежде всего к стартовой простоте, а не к гибкости архитектуры. Публичная витрина в этом случае зависит от шаблонного и runtime-слоя самой CMS, поэтому развитие frontend обычно происходит внутри ограничений конкретной темы, движка и способа рендеринга. Это усложняет ситуацию, когда требуется независимо изменять структуру маркетинговых страниц, переиспользовать одни и те же контентные сущности в разных разделах или выносить часть публичного слоя в отдельный технологический контур.

Интеграция редактирования и рендеринга действительно упрощает базовый preview и публикацию: редактор работает в той же системе, которая формирует страницу. Однако для рассматриваемой предметной области этого недостаточно. Если локализация, SEO, метаданные и публикационный процесс завязаны на тему и внутренний runtime CMS, то они начинают зависеть не столько от контентной модели, сколько от частных доработок слоя представления. В результате традиционный подход не исключает нужные функции, но делает их вторичными по отношению к связанному шаблонному контуру, тогда как в ВКР требуется архитектура, где редакторский и публичный слои могут развиваться отдельно.

Преимущества headless CMS для маркетинговой платформы

Headless CMS исходит из другой модели: система управления отвечает за хранение и редактирование структурированных данных, а отображение контента переносится в отдельный frontend-слой. Такой подход естественным образом соответствует API-first логике, в которой CMS предоставляет данные через программный интерфейс, а витрина или другой потребитель решает, как именно эти данные представить пользователю. Для CMS-ориентированной разработки это означает более явное разделение контентной модели, правил публикации и публичного интерфейса, что хорошо согласуется с современными model-driven и platform-independent подходами в домене CMS [8—10].

Для маркетингового сайта эта модель важна не сама по себе, а как способ перевести ключевые требования предметной области в архитектурные свойства системы. Когда frontend не встроен в CMS, контентная модель может проектироваться относительно сущностей и редакторских сценариев, а не относительно

одного набора шаблонов. Это упрощает переиспользование контента, позволяет строить локализованную маршрутизацию и дает основу для отдельного публикационного контура, в котором обновление данных не требует ручной правки публичной витрины [11—13].

При этом headless подход не снимает инженерные задачи автоматически. В частности, preview, управление метаданными и публикация должны проектироваться явно, так как CMS не рендерит итоговую страницу самостоятельно. Это усложняет начальную интеграцию, но одновременно делает публикационный контур явным и проверяемым: редакторский слой, API, frontend-сборка и публичная выдача перестают смешиваться в одном runtime-сценарии. Для мультиязычных контентных платформ это особенно существенно, поскольку локаль влияет не только на тексты интерфейса, но и на маршруты, метаданные и доступность данных в разных языковых версиях [11; 14—16].

Для формализации выбора архитектурного подхода под задачу маркетинговой витрины целесообразно сопоставить оба варианта по критериям, которые прямо следуют из предметной области и целевого результата ВКР (таблица 1.2).

Таблица 1.2 – Сравнение традиционной CMS и headless CMS по критериям маркетингового сайта

Критерий	Традиционная CMS	Headless CMS	Значение для задачи ВКР
Зависимость frontend-слоя от CMS	Шаблоны и рендеринг обычно встроены в CMS; развитие витрины зависит от ее theme- и runtime-слоя.	Публичный frontend развивается отдельно и использует CMS как источник данных по API.	Для независимого развития витрины и редакторского контура предпочтительно слабое связывание представления с CMS.
Пригодность к мультиязычности	Локализация часто расширяется на уровне шаблонов, плагинов и отдельных страниц.	Локаль можно проектировать на уровне данных, API, маршрутов и метаданных как единый контур.	Для обязательной схемы ru/en требуется архитектурная, а не точечная локализация.
Повторное использование контента	Контент нередко привязан к конкретным шаблонам и страницам сайта.	Структурированные сущности и блоки проще использовать повторно в разных страницах и каналах.	Маркетинговые страницы, статьи, проекты и вакансии требуют общей контентной модели, а не набора изолированных шаблонов.

Продолжение таблицы 1.2

Критерий	Традиционная CMS	Headless CMS	Значение для задачи ВКР
Preview и управляемая публикация	Базовый preview проще, так как редактирование и рендеринг находятся в одном контуре; публикация тесно связана с runtime CMS.	Preview требует отдельной интеграции с витриной, но публикацию можно оформить как явный pipeline между CMS и публичным слоем.	Для диплома важен не только просмотр черновика, но и формализованный путь “черновик -> проверка -> публичная витрина”.
Управляемость SEO и метаданных	Базовые настройки часто доступны из коробки, но их логика сильнее связана с темой и шаблонами.	SEO-поля и метаданные нужно моделировать явно, после чего ими можно управлять как частью контентной сущности.	Для маркетингового сайта критично редакторски управляемое SEO: canonical, meta и Open Graph без правки frontend-кода.
Удобство для воспроизводимого deployment-контура	Стартовый deploy проще как одного приложения, но публичная выдача и администрирование остаются в одном runtime-контуре.	Контуры CMS и витрины разводятся, что требует интеграции, но упрощает versioned deployment и изоляцию публикации.	Для rebuild-based публикации важнее предсказуемый и воспроизводимый pipeline, чем минимальное число исполняемых компонентов.

Матрица показывает, что традиционный подход сохраняет преимущество в стартовой простоте и во встроенности базового preview, однако по критериям, определяющим предметную область данной ВКР, решающими оказываются другие свойства: независимость frontend-слоя, сквозная мультиязычность, переиспользование контента, редакторски управляемое SEO и воспроизводимая публикация. Эти требования могут быть реализованы и поверх традиционной CMS, но для нее они обычно выступают как надстройки над связанным шаблонным контуром. Для headless CMS те же свойства являются прямым следствием разделения контентного и презентационного слоев. Следовательно, выбор headless CMS в рамках данной работы является не данью технологическому тренду, а рациональным ответом на критерии, заданные архитектурой корпоративного маркетингового сайта.

1.3 ОБОСНОВАНИЕ ВЫБОРА STRAPI И ASTRO

Strapi как ядро управления контентом

После выбора headless-подхода необходимо обосновать уже не только сам класс архитектуры, но и конкретную технологическую связку. Для рассматриваемой ВКР выбор Strapi, Astro и монорепозитория Nx + pnpm определяется не полнотой их возможностей как таковой, а тем, насколько они закрывают критерии, непосредственно вытекающие из предметной области маркетингового сайта (таблица 1.3).

Таблица 1.3 – Критерии выбора стека для CMS-first маркетинговой платформы

Критерий выбора	Почему он важен	Как его закрывает выбранный стек
Готовая CMS-инфраструктура	Цель ВКР состоит в разработке системы управления контентом для маркетинговой команды, а не в создании административной панели и базовых редакторских механизмов с нуля.	Strapi предоставляет административный интерфейс, контентные API, Draft & Publish, Preview, media-layer и базовую ролевую модель, что позволяет сосредоточить разработку на предметной модели и публикационном контуре [17].
Структурированные content types, roles и permissions	Страницы, статьи, проекты и вакансии имеют разный жизненный цикл, а локализация ru/en и разделение ролей редактора, маркетолога и администратора должны быть частью архитектуры данных, а не набором частных доработок шаблонов.	Strapi позволяет моделировать самостоятельные контентные сущности, локализованные записи и разграничение доступа на уровне CMS, то есть там, где фактически происходит редакторская работа [17].
Предсобранный frontend-контур для маркетинговой витрины	Для контентно-ориентированных страниц важны предсказуемая отдача, индексируемость и минимальная зависимость публичной витрины от runtime-логики CMS.	Astro ориентирован на предварительную сборку маршрутов и страниц, поэтому основной публичный слой может публиковаться как заранее подготовленная витрина, а рендеринг по запросу использоваться только там, где он действительно необходим [18].
Формализованный сценарий публикации	При разделении CMS и витрины публикация должна быть воспроизводимым процессом, а не ручной синхронизацией данных и frontend-кода.	В связке Strapi + Astro изменение контента логически переводится в схему webhook -> rebuild: CMS фиксирует публикационное событие, а frontend пересобирает публичную витрину на основе обновленных данных [19].

Продолжение таблицы 1.3

Критерий выбора	Почему он важен	Как его закрывает выбранный стек
Согласованное развитие CMS и frontend	Контентная схема, API-контракт и рендеринг блоков изменяются совместно; раздельное сопровождение повышает риск рассинхронизации между двумя частями системы.	Монорепозиторий Nx + nxpm позволяет хранить оба приложения в одном workspace, выполнять атомарные изменения, использовать единый набор зависимостей и согласованные команды сборки, что особенно важно для общего контентного контракта [20].

Решающим аргументом в пользу Strapi является не абстрактная популярность платформы, а то, что она закрывает сразу два базовых критерия: наличие готовой CMS-инфраструктуры и возможность строить управление на основе структурированной контентной модели. Согласно официальной документации Strapi 5, система поддерживает контентные типы, административное управление содержимым, Draft & Publish, международную локализацию, Preview, Role-Based Access Control и работу через контентные API [17]. Для темы данной ВКР это принципиально, поскольку позволяет сосредоточить инженерные усилия на моделировании страниц, статей, проектов, вакансий и связанных редакторских сценариев, а не на разработке базовой CMS-обвязки.

Не менее важно, что Strapi работает со структурированными сущностями, а не с жестко зашитым набором шаблонов. Для маркетингового сайта это означает возможность развести в модели данных самостоятельные типы контента, локализованные версии записей и ролевое разграничение доступа. В результате требования к idn, roles/permissions и управляемому редакторскому циклу закрепляются на уровне CMS-ядра, то есть в том слое, где действительно создается и публикуется контент, а не в виде поздних надстроек над frontend-частью.

Astro как публичная витрина и Nx + nxpm как среда согласованного развития

Astro выбран не просто как еще один frontend-framework, а как средство закрыть критерий предсобиранного публичного контура. Официальная документация указывает, что по умолчанию страницы и маршруты в Astro предварительно рендерятся во время сборки, а переход к рендерингу по запросу выполняется только для специально выделенных сценариев [18]. Для маркетинговой витрины это соответствует характеру нагрузки: основной массив страниц является кон-

тентным и должен отдаваться как уже подготовленное представление, пригодное для индексирования и не зависящее от шаблонного runtime самой CMS.

Из этого архитектурного решения непосредственно вытекает и схема webhook -> rebuild. Если Strapi отвечает за хранение и публикацию контента, а Astro — за выпуск предсобранной витрины, то обновление публичного слоя не может сводиться к мгновенному рендерингу внутри CMS. Оно должно оформляться как явный публикационный переход от события в CMS к пересборке frontend-слоя. Именно поэтому для статического сценария совместного использования Strapi и Astro в официальном руководстве также предполагается пересборка по webhook [19]. В контексте данной ВКР это не частная техническая деталь, а логическое следствие выбранной CMS-first архитектуры.

Монорепозиторий Nx + nxpm дополняет эту связку на уровне сопровождения. Официальная документация Nx подчеркивает преимущества общего репозитория для нескольких приложений, включая атомарные изменения и единый набор инженерных процедур [20]. Для рассматриваемого проекта это важно не само по себе, а потому, что CMS и витрина развиваются как две части одной системы: изменение контентной схемы влияет на API-контракт, а изменение контракта влияет на frontend-рендеринг и публикационный процесс. Следовательно, Nx + nxpm полезен именно как средство согласованного развития Strapi- и Astro-контуров в одной кодовой базе, с меньшим риском рассинхронизации между моделью данных и публичной витриной.

Таким образом, выбор Strapi, Astro и Nx + nxpm обоснован через связанный набор критериев: готовую CMS-инфраструктуру, структурированную контентную модель, предсобранный frontend-контур, воспроизводимую публикацию и согласованное сопровождение двух приложений. На этой основе далее можно формировать требования к разрабатываемой системе, не смешивая аналитическую аргументацию с деталями проектной реализации второй главы.

1.4 ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К СИСТЕМЕ

Функциональные требования

Для аналитической главы недостаточно просто перечислить требуемые возможности. Чтобы требования действительно выступали входом в проектирование, каждое из них должно быть связано с источником возникновения, целевым контуром реализации и способом последующей проверки. В рассматриваемой

ВКР функциональные требования распределяются между двумя связанными пользовательскими контурами: редакторским, в котором контент создается, проверяется и публикуется, и публичным, в котором результат должен отдаваться как управляемая и предсказуемая маркетинговая витрина.

Таблица 1.4 – Трассировка функциональных требований к системе

Требование	Источник требования	Контур реализации и верификации
Управление ключевыми контентными сущностями и глобальными данными сайта	Частые контентные изменения, различие жизненных циклов страниц, статей, проектов, вакансий и общих данных витрины	Реализация раскрывается в главе 2 через общую архитектуру, модель данных CMS и CMS-управляемые глобальные данные. Проверочный контур задается редакторским процессом, приемочной матрицей публичных маршрутов и пакетом тестовых доказательств для репрезентативных сущностей.
Самостоятельная сборка маркетинговых страниц из переиспользуемых блоков	Требование независимого редакторского цикла без ручной правки frontend-кода при типовых изменениях структуры страницы	Реализация раскрывается в подразделах главы 2 о page, Dynamic Zone и связке CMS-блоков с frontend-рендерингом. Проверка связывается с редакторским процессом подготовки и публикации страницы, а также с приемочными сценариями для репрезентативных CMS-страниц и их preview.
Поддержка ru/en для ядра публичной витрины и контента с маршрутами /:locale/*	Мультиязычность публичного контента, необходимость локализованных маршрутов, метаданных и редакторских сценариев; зафиксированная граница объема для карьерного модуля	Реализация раскрывается в главе 2 через локализуемую модель данных, маршрутизацию /:locale/* и CMS-управляемые глобальные данные. Проверка должна вестись по приемочной матрице локализации и пакету тестовых доказательств; при этом карьерный модуль входит в общий ru/en через locale-prefixed production routes и проверяется тем же набором артефактов, хотя сохраняет собственный прикладной сценарий отклика.
Управляемые SEO, Open Graph и sitemap	Требование поисковой видимости и редакторского управления метаданными без правки шаблонов витрины	Реализация раскрывается в главе 2 через SEO-модель CMS, единый контур метаданных и генерацию sitemap. Проверка связывается с приемочной матрицей SEO/sitemap и пакетом тестовых доказательств для публичных и preview-маршрутов.
Защищенный preview mode для чернового контента	Необходимость проверить материал до публикации и развести draft- и public-состояния	Реализация раскрывается в главе 2 через server-side preview на стороне frontend и ограничение публичного API опубликованным контентом. Верификация опирается на приемочную матрицу preview-сценариев, модель безопасности, редакторский процесс и пакет тестовых доказательств.

Продолжение таблицы 1.4

Требование	Источник требования	Контур реализации и верификации
Публикация через схему webhook -> rebuild	Требование воспроизводимой публикации для предсобранной витрины и разделение ролей между CMS и frontend	Реализация раскрывается в главе 2 через подразделы о публикации и развертывании. Верификация связывается с описанием контура публикации и развертывания, приемочной матрицей публикации и пакетом тестовых доказательств по webhook, сборке и публичному результату пересборки.
Разделение ролей, прав доступа и редакторских полномочий	Требование независимого редакторского цикла и отделение подготовки материала от его публикации и системного администрирования	Реализация раскрывается в главе 2 через подраздел о ролях и правах доступа. Проверка опирается прежде всего на формальную модель безопасности, редакторский процесс и артефакты контура синхронизации ролей; для этого требования не следует подменять формальную верификацию декларативным тезисом о наличии ролей.
Защищенные публичные формы для лидов и откликов	Предметная область включает маркетинговую коммуникацию и карьерный сценарий, где публичная форма должна приводить к управляемой прикладной записи	Реализация раскрывается в главе 2 через подразделы о карьерном модуле и валидации публичных форм. Проверка связывается с приемочной матрицей форм, пакетом тестовых доказательств и данными прикладных записей, появляющихся после валидной отправки.

Такая матрица показывает, что функциональные требования в рамках ВКР трактуются не как абстрактный перечень возможностей, а как трассируемый контракт между аналитической, проектной и проверочной частью. При этом характер верификации у требований различается: публичные и редакторские сценарии проверяются через приемочный контур и тестовые доказательства, тогда как ролевая модель, редакторский процесс и публикационный контур дополнительно требуют формального описания эксплуатационных правил и границ доступа.

Нефункциональные требования

Нефункциональные требования в данной работе определяются не только общими качественными ожиданиями от веб-системы, но и выбранной CMS-first архитектурой. Для маркетинговой платформы принципиально, что эксплуатационные свойства должны рассматриваться вместе с их доказуемостью: часть из них подтверждается тестами и сборочными артефактами, а часть фиксируется как

осознанная архитектурная граница, которую нельзя подменять недоказанными количественными обещаниями.

Таблица 1.5 – Трассировка нефункциональных требований к системе

Требование	Источник требования	Контур реализации и верификации
Ориентация на малую или среднюю корпоративную нагрузку и публикацию через пересборку	Масштаб предметной области и решение не усложнять систему под высокую нагрузку и сценарии реального времени, не входящие в тему ВКР	Реализация выражается в разделении CMS и витрины, предварительной сборке frontend и отсутствии контура доставки в реальном времени. Проверка здесь носит характер архитектурной границы: в главе 2 и тестовом контуре фиксируются сборочные и браузерные приемочные показатели, но не заявляется отдельная нагрузочная валидация высокого масштаба.
Воспроизводимый контур развертывания и публикации	Требование предсказуемого развёртывания и обновления витрины без ручного запуска frontend-команд редактором	Реализация раскрывается в главе 2 через подразделы о публикации и развертывании, а также через отдельные Docker-контуры frontend и CMS. Верификация связывается с описанием контура публикации и развертывания, приемочной матрицей публикации, сборочными артефактами и внешним подтверждением пути rebuild.
Базовый уровень безопасности для эксплуатационного контура	Необходимость защищать draft-контент, публичные формы, прикладные записи и административный доступ без ухода в отдельную тему информационной безопасности	Реализация раскрывается в главе 2 через preview-защиту, валидацию форм, ограничение публичного API и ролевую модель. Верификация связывается с security model, приемочной матрицей preview/forms и пакетом тестовых доказательств; дополнительные подтверждения вне этого контура не заявляются.
Сопровождаемость и согласованность развития системы	Требование развивать CMS и витрину как части одной платформы без рассинхронизации модели данных, маршрутов и представления	Реализация раскрывается в главе 2 через монорепозиторий Nx + nxpm, типизированный API-контракт, централизованную модель данных и переиспользуемые блоки. Верификация для этого свойства носит главным образом структурный характер и опирается на архитектурное описание, кодовую организацию и редакторский процесс, а не на отдельную числовую метрику.
Предсказуемая отдача, индексируемость и приемлемая производительность публичных страниц	Архитектурный акцент на предсборанной витрине и поисковой видимости маркетингового контента	Реализация раскрывается в главе 2 через Astro-сборку, единый SEO-контур и генерацию sitemap. Верификация связывается с приемочной матрицей SEO/sitemap, сборочными артефактами и браузерными приемочными измерениями; при этом корректно говорить именно о локальном приемочном уровне, а не о метриках Web Vitals в реальной эксплуатации.

Продолжение таблицы 1.5

Требование	Источник требования	Контур реализации и верификации
Доступность публичной витрины	Требование корректного пользовательского взаимодействия с ключевыми маршрутами и формами; нормативный ориентир WCAG 2.2 [21]	Реализация раскрывается в главе 2 через структурированную маршрутизацию, layout, метаданные и формы. Верификация связывается с приемочной матрицей доступности, браузерным аудитом и пакетом тестовых доказательств для репрезентативных публичных маршрутов.

Таким образом, нефункциональные требования в рамках данной ВКР задают не только желаемые свойства системы, но и формат инженерной честности при их интерпретации: воспроизводимые контуры сборки, публикации, безопасности и браузерных приемочных измерений должны подтверждаться фактическими артефактами, а не выводиться из общих ожиданий к современному web-стеку.

Ограничения и нецели проекта

Границы решения в данной работе задаются не как перечень случайно неохваченных функций, а как осознанное сокращение класса задач до CMS-first платформы для маркетингового контента. За пределы проекта вынесены прежде всего подсистемы, которые существенно меняют характер модели данных, публикационного контура, безопасности и эксплуатации:

- функции e-commerce, поскольку они требуют отдельного транзакционного домена: каталогов, корзины, заказов, оплат и связанной бизнес-логики, не относящейся к редакторскому управлению маркетинговым контентом;
- личные кабинеты и пользовательская аутентификация на стороне витрины, так как они переводят систему из плоскости CMS-first публикации в плоскость пользовательской платформы с собственным контуром идентификации, сессий и авторизации;
- сложные многошаговые процессы согласования, поскольку в рамках ВКР сознательно фиксируется более компактная и проверяемая схема draft -> preview -> publish с разграничением ролей, а не отдельный движок согласования;

- совместное редактирование в реальном времени, так как оно требует иной модели конкурентного доступа, разрешения конфликтов и синхронизации состояния.

К осознанным границам относится и мультязычный контур результата. Обязательной частью публичного ru/en в данной работе выступают ядро публичной витрины, page, article, project и vacancy с locale-prefixed production routes. Аналогично списковые маршруты разделов сохраняют маршрутно-управляемый слой SEO, тогда как CMS-управляемая поверхность метаданных осознанно ограничивается ключевыми страницами и детальными сущностями, а прикладные записи vacancy-application и lead-submission не включаются в локализуемый публичный контур.

Принципиально, что за рамки работы при этом не выводятся preview mode, базовые редакторские сценарии, защита публикации, разделение ролей и защита публичных форм. Именно они образуют минимально достаточный инженерный контур, позволяющий считать разрабатываемую систему не демонстрационным сайтом, а управляемой контентной платформой для корпоративной маркетинговой витрины.

1.5 ВЫВОДЫ ПО АНАЛИТИЧЕСКОЙ ГЛАВЕ

Проведенный анализ показывает, что корпоративный маркетинговый сайт следует рассматривать как контентную систему, а не как набор редко изменяемых страниц. Для такой предметной области существенны регулярные публикации, мультязычность, управление метаданными, сценарий предварительного просмотра и воспроизводимая доставка контента на публичную витрину. Эти свойства затруднительно обеспечивать последовательно в архитектуре, где слой управления контентом жестко связан со слоем отображения.

Сопоставление подходов к управлению контентом показывает, что headless CMS лучше соответствует задачам рассматриваемой предметной области, поскольку позволяет разделить контентную модель и frontend-представление, использовать предварительную сборку маркетинговых страниц и выстроить публикационный контур через API и webhook -> rebuild. В этом контексте выбор Strapi обоснован как выбор CMS-ядра с поддержкой структурированных сущностей, локализации и редакторских возможностей, а выбор Astro — как выбор пуб-

личной витрины, ориентированной на предсобранный контент и контролируемую публикацию.

На этой основе в главе сформулированы функциональные и нефункциональные требования к системе, причем они оформлены как трассируемый набор условий: для каждого требования определены его источник, контур проектной реализации и способ дальнейшей проверки. Тем самым в качестве входа во вторую главу выступает не просто перечень желаемых функций, а формализованный контракт между архитектурой, моделью данных, эксплуатационными решениями и приемочным контуром.

Следовательно, во второй главе внимание может быть сосредоточено на проектировании архитектуры, модели данных, редакторских сценариев, контуров публикации и развертывания, модели безопасности и тестирования как на средствах реализации и верификации сформированных требований. Зафиксированные ограничения при этом задают не слабые места проекта, а допустимые границы решения, внутри которых результат остается инженерно целостным и проверяемым.

ГЛАВА 2. ПРОЕКТНАЯ ЧАСТЬ

Во второй главе рассматривается, каким образом требования, сформулированные в первой главе, переводятся в реализованную систему управления контентом на базе Strapi и Astro. Если в аналитической части были зафиксированы необходимость независимого редакторского цикла, структурированной модели данных, локализованной витрины ru/en, управляемых метаданных, preview и воспроизводимой публикации, то далее показывается, как эти требования закреплены в архитектуре приложений, модели данных CMS, page builder-контуре и frontend-слое.

Описание опирается только на те решения, которые подтверждаются структурой репозитория, конфигурацией приложений, схемами контентных типов, маршрутами frontend-части и реализованными пользовательскими сценариями. Поэтому глава выстроена в последовательности “архитектура -> данные -> Dynamic Zone -> публичная витрина -> эксплуатационные контуры”: сначала фиксируется общий CMS-first каркас, затем раскрывается, как он превращается в управляемые контентные сущности, в конструктор страниц и в опубликованный storefront. Такой порядок позволяет рассматривать проектную главу не как план развития системы, а как описание уже существующего инженерного контура управления маркетинговым контентом.

2.1 ОБЩАЯ АРХИТЕКТУРА СИСТЕМЫ

Структура репозитория и роли приложений

Проект реализован как монорепозиторий Nx + pnpm, поскольку в рамках CMS-first системы контентная схема, API-контракт, frontend-рендеринг и публикационный контур изменяются совместно. В составе репозитория выделены два самостоятельных приложения: apps/cms на Strapi 5 и apps/front на Astro 6. Для каждого из них определены собственные Nx-цели разработки, сборки и проверки, поэтому CMS и публичная витрина сопровождаются как независимые исполняемые контуры, но остаются в одной кодовой базе. Практический эффект такого решения состоит в том, что изменение модели данных, маршрутов или публикационного механизма можно проводить атомарно, без рассинхронизации между backend и frontend.

С архитектурной точки зрения Strapi выступает ядром редакторской части системы. В нем сосредоточены контентные типы, отношения между сущностями, локализуемые поля, режим draft/publish, роли и публичный REST API. Astro, напротив, выполняет роль слоя публикации: получает данные из CMS, формирует маршруты витрины, рендерит страницы и обслуживает preview-сценарии. Тем самым разделение, обоснованное в первой главе, становится реализованной архитектурой: контент управляется в CMS, а публичное представление строится в отдельном frontend-контуре.

Интеграция между этими приложениями организована в двух взаимодополняющих формах. Во-первых, Strapi публикует OpenAPI-описание, а во frontend используется генерация типизированного клиента, что снижает риск расхождения между маршрутами backend и потребителями данных. Во-вторых, для global, home-page, page и карьерного модуля сохранен отдельный слой доступа к данным с явным управлением локалями, populate-параметрами и preview-режимом. Такое сочетание не является дублированием: типизированный клиент удобен для стабильных коллекционных сценариев, тогда как page builder- и preview-контур требуют более точного контроля над составом загружаемых данных. Сводно архитектура системы показана на рисунке 2.8 в приложении А.

Контур доставки контента

Контур доставки контента реализует центральное требование первой главы: публичная витрина должна зависеть от управляемых данных CMS, а не от ручной правки шаблонов. Для этого в системе закреплена схема Strapi -> API -> Astro: контент создается и проходит публикационный жизненный цикл в Strapi, после чего frontend запрашивает уже опубликованные данные и использует их для формирования публичных страниц. Для home-page, page, а также для разделов articles, projects и vacancies эта схема реализована в маршрутах с языковым префиксом. Тем самым маршрутная граница витрины следует принятому объему работы: маркетинговые страницы, редакционные разделы и карьерный сценарий публикуются как единый двуязычный публичный контур при общей CMS-модели данных.

На стороне frontend этот подход поддерживается конфигурацией Astro: подключены адаптер Node.js, модуль sitemap и абсолютный site-контракт через переменные окружения. Итоговая сборка формируется в standalone-режиме и служит основой для отдельного Docker-приложения витрины, которое далее раз-

ворачивается в Dokploy. При этом базовый режим работы остается статически ориентированным: основные публичные маршруты публикуются как заранее подготовленный результат, а серверная логика используется только там, где она действительно нужна, прежде всего для preview. Практический эффект заключается в предсказуемой публикации контента: CMS управляет данными и событиями публикации, а витрина отдает уже сформированное представление.

2.2 ПРОЕКТИРОВАНИЕ МОДЕЛИ ДАННЫХ CMS

Сущности, атрибуты и связи

Модель данных CMS выступает тем уровнем, на котором требования главы 1 превращаются в формализованные управляемые сущности. Вместо одного универсального “страничного” типа в Strapi разделены несколько контуров: редакционный маркетинговый контент, карьерный прикладной контур, глобальные данные витрины и записи, возникающие в результате действий внешнего пользователя. Поэтому в системе определены коллекции article, author, project, vacancy, vacancy-application, lead-submission, industry, job-role и page, а также single type-сущности global и home-page. Такой набор отражает не структуру отдельных шаблонов, а состав предметной области, которой должен управлять редактор.

Редакционный контур охватывает прежде всего page, home-page, article, author и project. Он отвечает за страницы витрины, публикации и кейсы, то есть за сущности, для которых критичны локализуемые slug, rich text-контент, media-атрибуты и управляемый жизненный цикл публикации. Карьерный контур организован вокруг vacancy, industry и job-role: в vacancy зафиксированы не только заголовки и описание, но и прикладные атрибуты кадрового сценария, включая связь с отраслью и ролью, формат работы, тип занятости, уровень и зарплатные диапазоны. Практический эффект такого разделения состоит в том, что маркетинговый и кадровый сценарии используют общее CMS-ядро, но не смешивают свои типы контента и правила его сопровождения.

Отдельный слой образуют прикладные записи, создаваемые не редактором, а пользователем витрины. Сущность vacancy-application хранит ссылку на вакансию, персональные данные кандидата, файл резюме, отметку согласия, момент отправки, источник и внутренний HR-статус. Сущность lead-submission, в свою очередь, обслуживает маркетинговый сценарий и сохраняет имя, email, телефон, компанию, текст запроса, submittedAt, идентификатор формы, путь и

заголовок страницы, а также локаль отправки. За счет этого CMS используется не только как хранилище страниц и публикаций, но и как единая точка фиксации прикладных результатов работы витрины.

На уровне связей модель включает отношение manyToMany между статьями и авторами, отношение manyToOne от вакансии к отрасли и роли, а также отношение manyToOne от отклика к вакансии. Для article, author, project, vacancy, industry, job-role, global, home-page и page включены draft/publish и i18n, поскольку для этих сущностей требуется редакторская подготовка, локали ru/en и отдельные состояния черновика и публикации. Для vacancy-application и lead-submission такие механизмы намеренно не используются: эти записи фиксируют факт отправки формы, а не объект редакторского контроля. Следовательно, сама структура модели данных уже проводит границу между управляемым контентом и прикладными событиями. Сводная модель сущностей и их связей приведена на рисунке 2.9 в приложении Б, а жизненный цикл локализуемой контентной единицы с режимом draft/publish — на рисунке 2.10 в приложении В.

SEO-модель и глобальные сущности

Важную роль в этой модели играют две single type-сущности: global и home-page. Первая закрывает требование о CMS-управляемых общих данных витрины: в global вынесены название компании, описание, логотип, навигация, primary и secondary СТА, колонки футера и копирайт. Благодаря этому layout уровня Header/Footer перестает зависеть от локальных frontend-констант и получает тот же редакторский статус, что и основной контент страницы.

Сущность home-page решает другую архитектурную задачу: главная страница включается в ту же управляемую модель, что и обычные page-страницы. Она хранит описание, набор блоков и вложенный SEO-компонент, поэтому главная витрина редактируется не как исключение в коде, а как самостоятельная запись в CMS. Далее эта логика расширяется на единый SEO-контур: компонент shared.seo используется не только в home-page и page, но и в сущностях article, project и vacancy. Он задает metaTitle, metaDescription, canonicalURL, поля Open Graph, изображение для предпросмотра и признак noIndex. В результате frontend получает один и тот же контракт метаданных для главной страницы, маркетинговых slug-страниц и ключевых детальных сущностей. Практический эффект состоит в том, что витрина может строить SEO-слой централизованно, не дублируя схему метаданных в каждом типе маршрута.

2.3 КОНСТРУКТОР СТРАНИЦ НА ОСНОВЕ DYNAMIC ZONE

Сущность page и набор блоков

Коллекция page выступает основным редакторским интерфейсом для независимой сборки маркетинговых страниц. Каждая запись объединяет локализуемый slug, SEO-метаданные и массив блоков, а структура страницы хранится не в route-файле frontend, а в Dynamic Zone на стороне CMS. Именно здесь требование первой главы о редактировании типовой маркетинговой страницы без правки frontend-кода получает практическую реализацию: редактор меняет не только тексты, но и композицию страницы как часть одной сущности.

Для page и home-page используется единая библиотека блоков, в которую входят hero, rich-text, feature-cards, cta, preview-list, logo-cloud, testimonials, stats, feature-highlight, faq, process-timeline, quote, checklist, content-columns, numbered-points и lead-form. Блоки оформлены как отдельные Strapi-компоненты и используют вложенные shared-компоненты для карточек, ссылок, FAQ-элементов, шагов процесса, статистики и других переиспользуемых частей модели данных. Такое решение переводит page builder из набора одноразовых секций в устойчивый словарь редакторских конструкций, одинаково применимый к главной и внутренним маркетинговым страницам.

Особенно показателен блок lead-form: он задает не только визуальную секцию, но и поля прикладного сценария, включая имя формы, тексты согласия и сообщения после отправки. За счет этого page builder охватывает не только презентационную, но и конверсионную функцию витрины. Практически такой редакторский контур виден на рисунке 2.1: в одной сущности одновременно задаются slug, последовательность блоков, SEO-поля и точка входа в preview-сценарий.

Связь CMS-блоков с frontend-рендерингом

На стороне frontend Dynamic Zone поддерживается общим renderer-слоем, который принимает массив блоков и передает каждый элемент в специализированный компонент на Astro или React. Тем самым frontend не содержит ручного описания конкретной маркетинговой страницы: route-файл отвечает только за получение записи, маршрутный контекст и метаданные, а фактическая композиция страницы определяется данными CMS. Это и делает page builder полноценным механизмом публикации, а не только удобством админ-панели.

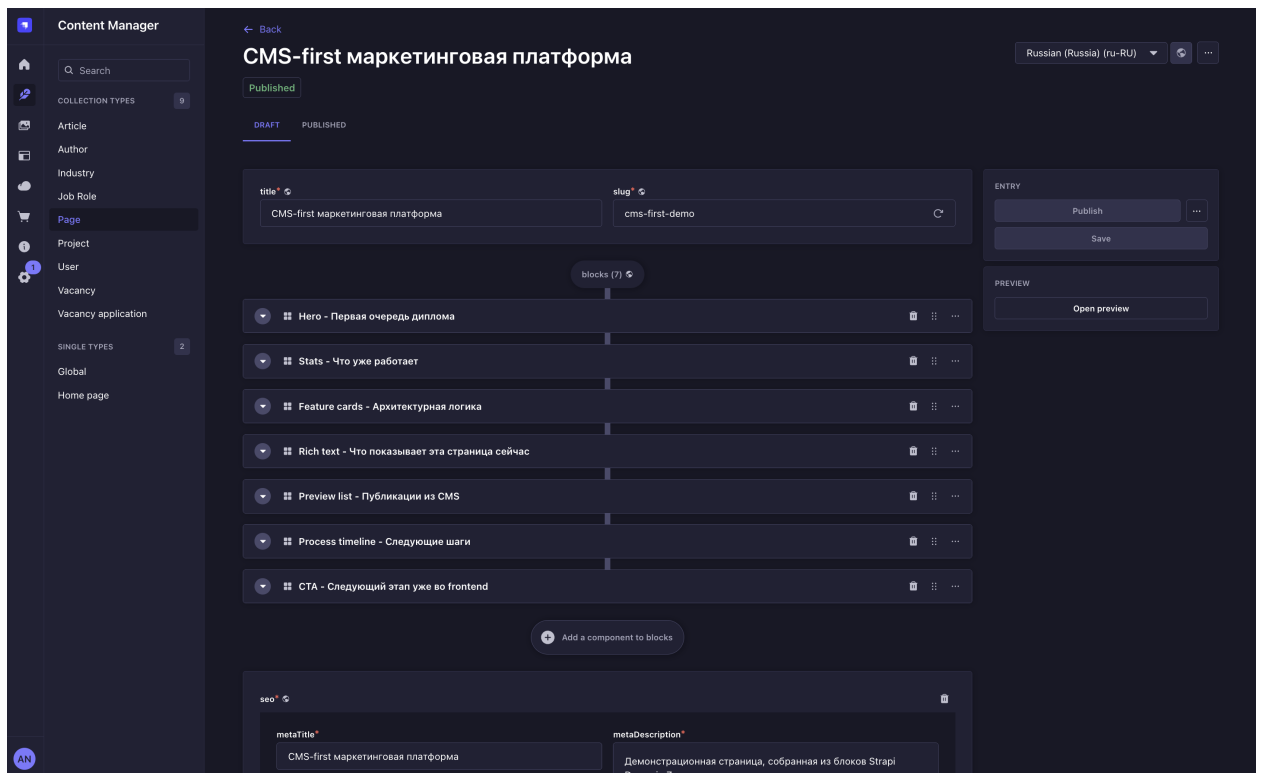


Рисунок 2.1 – Редактирование CMS-страницы в Strapi: slug, Dynamic Zone, SEO и доступ к preview

Практический эффект такого маппинга особенно заметен в двух случаях. Для блока preview-list реализована связка с разделами articles, projects и vacancies, поэтому CMS-страница может включать динамические анонсы существующих коллекций без отдельной ручной раскладки в шаблоне. Для блока lead-form frontend дополнительно передает путь, заголовок и локаль текущей страницы, что позволяет сохранять маркетинговый лид вместе с контекстом его происхождения. Следовательно, Dynamic Zone связывает модель данных с реальным пользовательским поведением витрины, а не ограничивается текстовым наполнением секций.

2.4 РЕАЛИЗАЦИЯ FRONTEND-ЧАСТИ НА ASTRO

Локализованные маршруты главной и CMS-страниц

Локализованный маршрутный слой frontend является прямым продолжением CMS-модели данных. Для главной страницы используются маршруты /ru/ и /en/, для page — маршруты вида /:locale/:slug/, а корневой адрес / выполняет редирект на /ru/ как на базовую точку входа в витрину. Во время сборки frontend получает из CMS список локалей и набор доступных slug для каждой локали,

после чего route-файлы формируют локализованный путь, SEO-метаданные и передают массив блоков в общий `renderer`. Тем самым адрес, контент и метainформация страницы выводятся из одной и той же записи CMS, а не собираются из разрозненных frontend-констант.

С инженерной точки зрения это означает, что главная витрина и маркетинговые slug-страницы уже работают как единая CMS-first витрина: появление или изменение страницы определяется публикацией записи в Strapi, а не созданием нового route-файла. Такой способ построения мультиязычного маршрутного контура соответствует как встроенной i18n-модели Astro, так и рекомендациям по публикации локализованных версий страниц для поисковых систем [22; 23]. На рисунке 2.2 показана английская версия главной страницы по маршруту `/en/`; тот же принцип адресации с языковым префиксом используется и для управляемых CMS-страниц с маршрутами вида `/:locale/:slug/`.

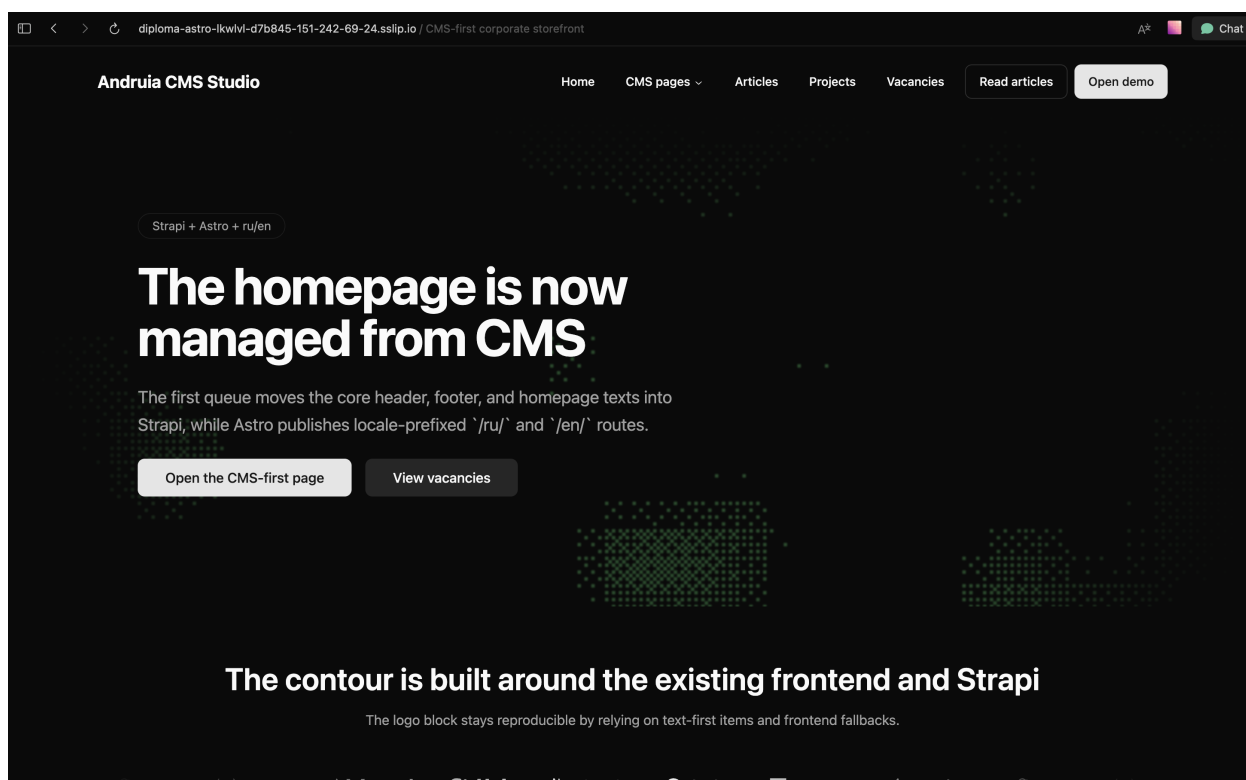


Рисунок 2.2 – Публичная витрина в маршруте с языковым префиксом `/en/`, опубликованная на основе данных из CMS

Статьи, проекты и карьерный модуль

Frontend не ограничивается page builder-маршрутами и тем самым завершает превращение CMS из хранилища страниц в источник всей публичной витрины. Для `articles` и `projects` реализованы отдельные коллекционные разде-

лы, встроенные в маршрутный контур с языковым префиксом `/:locale/articles/*` и `/:locale/projects/*`. Списковые страницы этих разделов используют сгенерированный типизированный клиент, а детальные страницы загружают полную сущность по slug. Практический эффект такого решения состоит в том, что одна и та же CMS-модель обеспечивает как произвольные маркетинговые страницы, так и повторяемые редакционные форматы публикаций и кейсов внутри общего storefront-контура.

Карьерный модуль при этом не остается изолированным от storefront-контура, а встраивается в тот же locale-prefixed маршрутный слой. Каталог использует `VacancyExplorer` с фильтрацией по отрасли, роли, локации, формату работы, типу занятости и уровню, а детальная страница вакансии отображает атрибуты записи и подключает форму отклика. Тем самым frontend обслуживает уже не только редакционные материалы, но и кадровый пользовательский сценарий, в котором `Strapi` остается источником данных как для самих вакансий, так и для связанных таксономий. Соответствующая публичная точка взаимодействия показана на рисунке 2.3: детальная страница одновременно отображает данные вакансии и форму, которая затем передает отклик в `vacancy-application`. Важно, что в итоговом дипломном score это трактуется уже не как локализуемая модель без production-адресации, а как полноценный двуязычный маршрутный контур `/:locale/vacancies/*`: локализация вакансий, отраслей и ролей используется и в модели данных, и в preview, и в опубликованных публичных URL кадрового сценария.

The screenshot shows a job listing for a **Frontend React Engineer** in Saint-Petersburg. The page is part of the **Andruia CMS Studio** interface. It features a navigation bar with links like **Главная**, **CMS-страницы**, **Статьи**, **Проекты**, **Вакансии**, **К статьям**, and **Открыть демо**. Below the navigation, there are filters for **Информационные технологии** and **Инженер разработки**. The job title **Frontend React Engineer** is prominently displayed, followed by the location **Санкт-Петербург**.

Key details about the job are listed in a grid:

- ФОРМАТ:** Удаленно
- ЗАНЯТОСТЬ:** Полная занятость
- УРОВЕНЬ:** Middle
- ЗАРПЛАТА:** 200 000 - 290 000 RUB

The **Описание вакансии** (Job Description) section includes **Требования:** (Requirements) and **Будет плюсом:** (It will be a plus):

- Требования:**
 - Опыт коммерческой разработки от 2–3 лет
 - Уверенное владение TypeScript, React, Next.js
 - Понимание клиент-серверного взаимодействия (REST / API)
 - Опыт работы с UI-библиотеками и их кастомизации
 - Понимание принципов компонентного подхода
 - Опыт работы с системами контроля версий (Git)
 - Базовое понимание работы Docker
- Будет плюсом:**
 - Опыт работы с NX (монорепозитории)
 - Опыт работы с Directus или другими headless CMS

On the right side, there is a **Отклик на «Frontend React Engineer»** (Respond to «Frontend React Engineer») form. It includes fields for **Имя и фамилия *** (Name and surname), **Email ***, **Телефон ***, and **Город** (City). There is also a **Сопроводительное письмо** (Cover letter) field with a character count (0/3000). Below the form, there is a **Резюме (PDF/DOC/DOCX, до 10MB) *** section with a file upload button and a checkbox for **Согласен(а) на обработку персональных данных для рассматривания отклика *** (Agree to the processing of personal data for consideration of the response).

Рисунок 2.3 – Детальная страница вакансии с подключенной формой отклика как прикладной публичный сценарий

Отдельно от карьерного сценария реализован маркетинговый прикладной контур для home-page и page. Если редактор добавляет в Dynamic Zone блок lead-form, frontend рендерит встроенную форму, валидирует ее на клиенте и затем передает данные в серверный маршрут `/api/lead-submissions`. Этот маршрут повторно проверяет payload, применяет honeypot и создает запись lead-submission в Strapi через технический серверный API token. За счет этого page builder доводится до полного практического цикла: CMS определяет не только состав секций страницы, но и сценарий получения маркетингового лида с сохранением контекста страницы и локали.

Таким образом, первая половина проектной главы выстраивает непрерывную линию от архитектурного разделения Strapi и Astro к предметной модели данных, затем к Dynamic Zone как редакторскому механизму сборки страницы и далее к frontend как к слою публикации этого контента в виде публичной витрины. На этой основе дальнейшие разделы главы уже раскрывают сквозные свойства системы — мультиязычность, preview, SEO и публикационный контур

— как продолжение уже сформированной архитектурной логики, а не как набор изолированных функций.

2.5 МУЛЬТИЯЗЫЧНОСТЬ

Локали в модели данных и в маршрутизации

Контур мультиязычности нужен системе для двух задач, сформулированных в первой главе: редактор должен вести контент в ru/en, а публичная витрина должна публиковать локализованные маршруты, метаданные и базовые элементы навигации как часть единой CMS-first модели. Поэтому во frontend зафиксированы две публичные локали ru и en, а для CMS определено отображение ru -> ru-RU, en -> en. На уровне Strapi локализация включена для global, home-page, page, article, project, vacancy, author, industry и job-role. Тем самым локаль закрепляется не в отдельных текстовых вставках frontend, а в самой модели данных, включая slug и связанные поля.

Публичная витрина использует этот слой данных в трех взаимосвязанных точках. Во-первых, локали участвуют в генерации маршрутов основной витрины /ru/, /en/, /:locale/:slug/, а также локализованных разделов articles и projects. Во-вторых, layout вычисляет по активной локали атрибут html[lang], canonical и значение og:locale. В-третьих, локаль передается в preview-сценарий, поэтому редактор видит черновую версию именно той языковой записи, которую подготавливает к публикации. Реализация этого контура подтверждается маршрутным слоем frontend, сборочным выводом и smoke-проверками, которые фиксируют доступность /ru/, /en/, локализованных CMS-страниц, а также разделов articles/projects.

Граница охвата при этом проведена явно. Для карьерного модуля локализуются сущности vacancy, industry и job-role, а preview вакансий принимает явную локаль и обращается к соответствующей версии записи. Production- публикация карьерного сценария также переведена в обязательный контур /:locale/vacancies/* с языковым префиксом. В итоговом объеме работы это означает единый locale-prefixed публичный контур для маркетинговых страниц, редакционных разделов и карьерного сценария.

CMS-управляемые глобальные данные

Локализация витрины не сводится к отдельным страницам, поэтому в CMS выделен дополнительный глобальный слой. Layout запрашивает сущность global

с учетом активной локали и передает в Header и Footer навигацию, СТА, описание компании, колонки футера и другие общие данные сайта. Практический эффект этого решения состоит в том, что редактор управляет не только телом конкретной страницы, но и общей языковой конфигурацией основной витрины без правки frontend-кода.

Этот контур подтверждается кодом layout-слоя и загрузкой global-данных на уровне маршрутов для активной локали, а также браузерными и smoke-проверками репрезентативных маршрутов ru/en. Его текущая граница также зафиксирована явно: CMS-управляемыми являются общие данные основной витрины, тогда как часть интерфейсных текстов отдельного карьерного публичного контура остается заданной на уровне маршрутов и не заявляется как полностью редакторски локализованный слой.

2.6 PREVIEW И ЗАЩИЩЕННЫЙ ДОСТУП К ЧЕРНОВИКАМ

Серверный preview на стороне frontend

Контур preview нужен системе для разведения редакторской проверки и публичной публикации. Для маркетинговой витрины и карьерного модуля это принципиально, поскольку редактор должен видеть будущую страницу до перевода записи в состояние published, а публичный storefront при этом обязан оставаться привязанным только к опубликованному контенту.

Реализация построена как связанный серверный поток CMS -> frontend. Маршрут /api/preview принимает тип сущности, локаль, slug, статус и секретный параметр, проверяет PREVIEW_SECRET, выбирает нужный fetcher и перенаправляет пользователя либо на опубликованный адрес, либо на соответствующий маршрут вида /preview/*. Для выхода из режима предпросмотра используется /api/exit-preview. Со стороны Strapi Admin этот сценарий встроен в конфигурацию preview: при наличии PREVIEW_SECRET и SITE_URL CMS формирует допустимые preview-ссылки для home-page, page, article, project и vacancy. Для home-page используется отдельная цель home, а для остальных сущностей CMS извлекает slug и строит ссылку на точку входа frontend /api/preview. Последовательность этого сценария приведена на рисунке 2.11 в приложении Г.

Для хранения preview-сессии frontend использует отдельный httpOnly cookie. Доступ к preview-маршрутам открывается только после его установки. Запросы к CMS выполняются с секретным preview-заголовком и черновым

статусом, поэтому frontend получает draft-версию записи, не раскрывая ее через обычный публичный API. Практически этот поток подтвержден кодом preview-маршрутов и smoke-проверками отказа по неверному секрету. Отдельно зафиксированы установка cookie, корректный переход к published-адресу и наличие noindex на preview-страницах для всех поддерживаемых типов сущностей. Для вакансий при этом сохраняется та же граница score, что и в production: preview локализован, а публичный адрес использует тот же контур `/:locale/vacancies/*`.

Практический результат этого потока показан на рисунке 2.4: редактор получает доступ к черновой версии страницы прямо из интерфейса Strapi, а preview открывается как отдельный режим просмотра, связанный с конкретной записью и ее draft-состоянием.

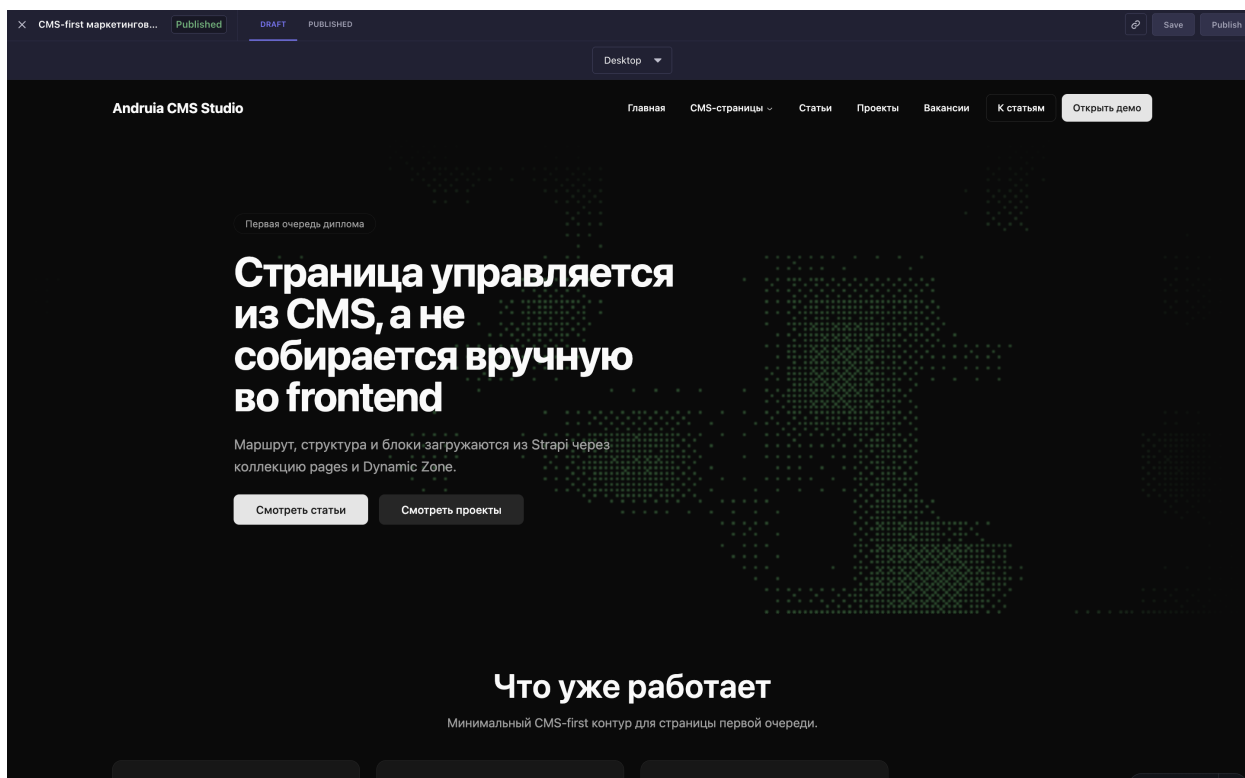


Рисунок 2.4 – Предпросмотр черновой версии CMS-страницы в интерфейсе Strapi

Ограничение публичного API опубликованным контентом

Чтобы preview не превращался в скрытый публичный draft-API, на стороне CMS реализовано дополнительное ограничение чтения контента. Middleware `enforce-published` для всех публичных GET `/api/*`, не относящихся к документации и не содержащих корректный `x-preview-secret`, принудительно выставляет `status=published`. Тем самым обычный запрос витрины или внешнего клиента по-

лучает только опубликованные данные, а чтение черновиков остается доступным лишь для серверного preview-контура.

Этот слой подтверждается кодом middleware и маршрутным поведением preview-маршрутов. Приемочные проверки отдельно воспроизводят отказ по неверному секрету и корректный доступ к draft-версии только через /preview/*. Граница здесь сознательно консервативна. Доступ к черновикам строится на общем секрете между CMS и frontend и не заявляется как отдельная пользовательская система аутентификации. Такое сужение публичной поверхности особенно важно для WCMS, поскольку исследования по безопасности подобных систем показывают, что избыточно открытые контентные эндпоинты и связанные с ними плагины становятся отдельным классом практических рисков [24; 25].

2.7 SEO, OPEN GRAPH И SITEMAP

Единый контур управляемых метаданных

Контур SEO/Open Graph нужен системе для того, чтобы поисковая видимость и предпросмотровые метаданные управлялись как часть контентной модели, а не оставались набором ручных правок во frontend. В реализованной системе эта задача решена через единый компонент shared.seo, который подключен к home-page, page, article, project и vacancy. При запросе данных frontend получает metaTitle, metaDescription, canonicalURL, значения Open Graph, изображение ogImage и признак noIndex. Далее функция buildSeoMetadata формирует итоговый набор метаданных и при необходимости дополняет его предсказуемыми fallback-значениями.

Такой резервный слой является частью принятой CMS-first архитектуры. Если отдельные SEO-поля не заполнены, frontend использует название сущности, ее описание, публичный путь маршрута и, для article/project, обложку как резервный og:image. После этого MainLayout централизованно рендерит title, description, canonical, og:*, twitter:* и robots; для preview-страниц дополнительно устанавливается noindex. Практически такой контур подтверждается кодом metadata builder и layout-слоя, browser audit для репрезентативных маршрутов, а также smoke-проверками title, canonical, Open Graph и robots как для public, так и для preview-страниц.

Граница охвата SEO-контура

Граница этого контура зафиксирована намеренно уже на уровне модели и маршрутов. CMS-управляемые метаданные охватывают главную страницу, маркетинговые slug-страницы и детальные маршруты article, project и vacancy. Следовательно, страницы `/:locale/articles/:slug/`, `/:locale/projects/:slug/` и `/:locale/vacancies/:slug/`, а также их preview-аналоги используют тот же управляемый через CMS SEO-контракт, что и `home-page/page`. Для вакансий это особенно важно: карьерный модуль входит в тот же публичный SEO-контур и не требует отдельной адресной модели.

В то же время списковые страницы `/:locale/articles/`, `/:locale/projects/` и `/:locale/vacancies/` не получают отдельной редакторской настройки через CMS. Для них заголовки, описание и маршрутная семантика формируются на уровне файлов маршрутов и затем проходят через тот же `buildSeoMetadata`- и `layout`-механизм. Это означает, что метаданные списковых страниц формируются на уровне маршрутов: единый рендер метатегов охватывает всю витрину, но редакторски управляемая поверхность намеренно ограничена ключевыми страницами и детальными страницами сущностей. Такая граница подтверждается самими `route`-файлами списков `articles/projects/vacancies` и не должна подменяться формулировкой о “полностью CMS-управляемом SEO” для всех публичных разделов.

Конфигурация `frontend` дополнительно подключает `@astrojs/sitemap`, а абсолютный адрес сайта задается через `SITE_URL/PUBLIC_SITE_URL`. В результате `canonical`-ссылки и `sitemap` строятся из единого внешнего URL-контракта. Этот слой подтверждается конфигурацией `Astro`, сборочным выводом `sitemap-index.xml/sitemap-0.xml` и приемочными проверками публичных маршрутов. Его текущая граница также определена явно: `sitemap` публикует основную витрину, редакционные разделы и карьерный модуль как единый двуязычный набор `locale-prefixed` маршрутов без `legacy URL` без префикса.

Публикационный контур и пересборка frontend

Публикационный контур нужен для того, чтобы редакторское действие `publish` в `Strapi` приводило к воспроизводимому обновлению предсобранной витрины, а не к ручному запуску `frontend`-сборки. В архитектуре с отдельными CMS и витриной это обязательный эксплуатационный слой: после публикации запись должна не только стать доступной по API, но и попасть в новый `build` публичного `frontend`-приложения.

Реализация выстроена как версионизируемая точка входа CMS с внешним платформенным исполнителем. Во время bootstrap backend синхронизирует автоматически регистрируемый webhook в `strapi_webhooks` и, соответственно, в разделе Settings -> Webhooks административной панели. Этот webhook имеет детерминированное имя, подписан только на `entry.publish` и `entry.unpublish` и указывает не на внешний webhook пересборки напрямую, а на версионизируемый маршрут `${PUBLIC_URL}/api/rebuild`. Если `FRONTEND_REBUILD_HOOK_TOKEN` задан, Strapi добавляет `x-rebuild-token`; контроллер `POST /api/rebuild` валидирует его и уже серверно вызывает внешний `DOKPLOY_DEPLOY_WEBHOOK_URL`, передавая `branch` и `repository` через `DOKPLOY_DEPLOY_BRANCH` и `DOKPLOY_DEPLOY_REPOSITORY`. Тем самым публикация оформлена как схема Strapi -> автоматически регистрируемый webhook -> CMS `/api/rebuild` -> Dokploy deploy webhook, а не как неформальная ручная связка между редактором и платформой.

Если `PUBLIC_URL` или `DOKPLOY_DEPLOY_WEBHOOK_URL` не заданы, CMS не пытается эмулировать внешний deployment: при bootstrap фиксируется предупреждение, а webhook остается неактивным. Эта граница принципиальна для академически корректного описания системы: в репозитории зафиксированы CMS-маршрут публикации и env-контракт, но не текущее состояние платформы Dokploy. Практически контур подтвержден тремя слоями доказательств: кодом `publication-webhook.ts` и `/api/rebuild`, локальным появлением записи в `strapi_webhooks` при запуске Strapi, а также внешней стендовой валидацией от 1 июня 2026 года, где был воспроизведен полный путь `publish/unpublish` -> `webhook` -> `Dokploy rebuild/redeploy` -> обновление Astro-витрины.

Практически этот слой настройки виден на рисунке 2.5: автоматически регистрируемый webhook Frontend rebuild hook зарегистрирован в Settings -> Webhooks, указывает на внешний URL `rebuild-hook` и подписан на события `publish/unpublish` для сущностей типа Entry.

На стороне frontend этот сценарий опирается на отдельный пакет развертывания `apps/front/Dockerfile`, где `build-stage` формирует standalone output Astro, а `runtime-stage` запускает серверное приложение на порту 4321. Именно это отдельное frontend Docker-приложение пересобирается и повторно развертывается в Dokploy. Публичные маршруты при этом сохраняют модель предварительной сборки, sitemap генерируется как часть build-контура, а серверная логика

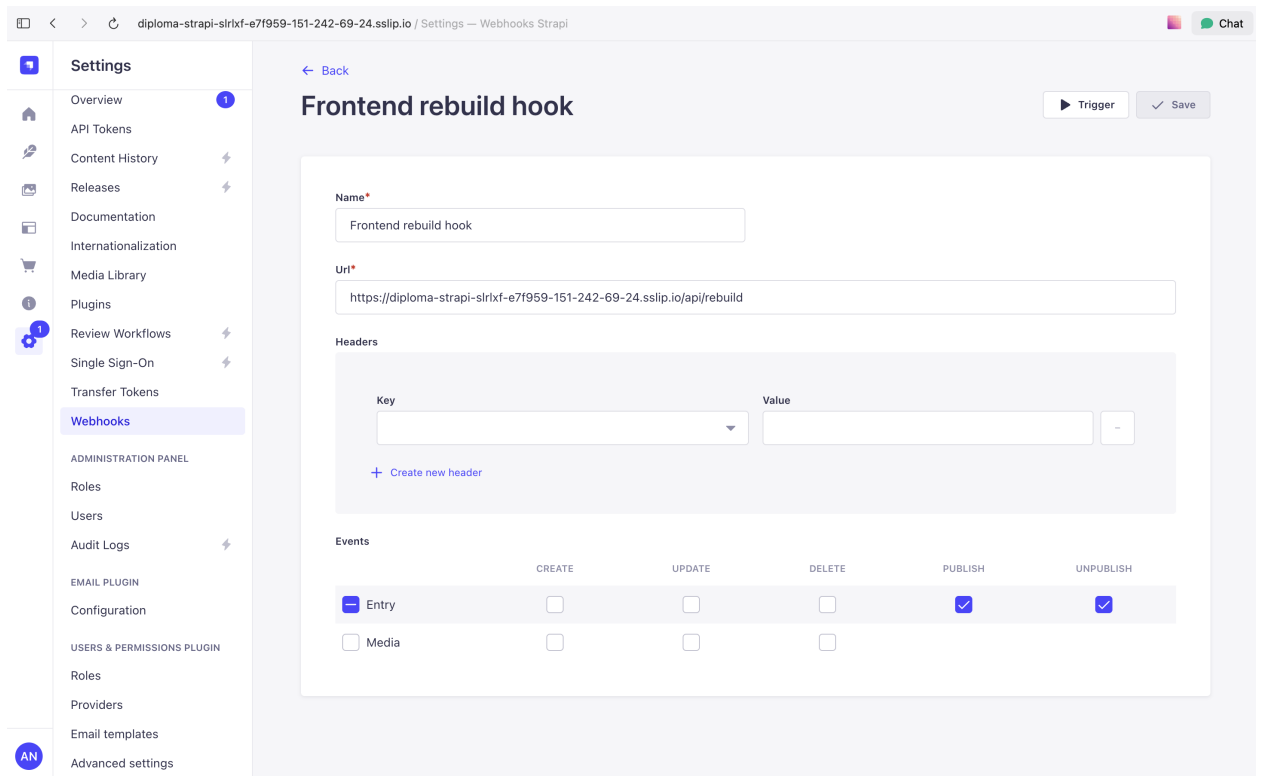


Рисунок 2.5 – Автоматически регистрируемый webhook пересборки в Strapi: целевой URL и подписка на entry.publish/entry.unpublish

используется точно для preview. Следовательно, публикация в рамках ВКР описывается как зрелый эксплуатационный контур без ручного участия редактора в build/deploy-процедурах и без утверждения, что вся runtime-конфигурация Dokploy зафиксирована внутри репозитория.

Архитектура развертывания frontend и CMS

Контур развертывания нужен системе для воспроизводимого разведения двух исполняемых ролей: публичной витрины и CMS-ядра. В production-модели, ориентированной на Dokploy, frontend и CMS разворачиваются как два независимых Docker-приложения. Frontend собирается из apps/front/Dockerfile, формирует standalone output Astro и выполняется как отдельное Node.js-приложение. CMS собирается из apps/cms/Dockerfile и запускает production build Strapi. Такое разделение соответствует фактической архитектуре проекта: витрина и CMS обслуживаются независимо, но остаются связаны единым публикационным контуром. На платформенном уровне это выражается как отдельные сервисы astro, strapi и pg в Dokploy, что показано на рисунке 2.6.

Для CMS в репозитории дополнительно зафиксирован переносимый runtime-контур на базе Docker Compose. Он включает Dockerfile, compose.yml,

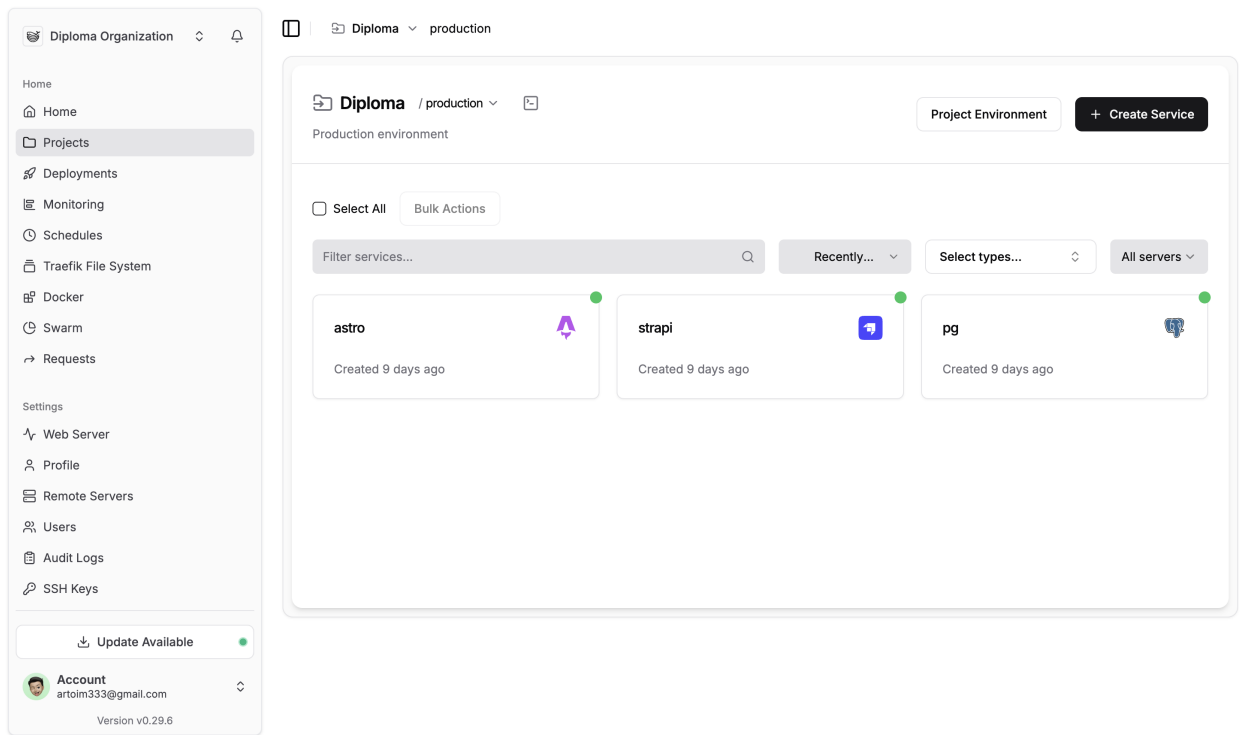


Рисунок 2.6 – Рабочее окружение в Dokploy с отдельными сервисами astro, strapi и pg

сервис cms с приложением Strapi, сервис cms-db с PostgreSQL и persistent volume для uploads. Этот слой нужен не как замена Dokploy, а как фиксируемое в репозитории описание минимально необходимой инфраструктуры CMS. На практике модель развертывания выражается через отдельные пакеты развертывания frontend и CMS, а также через согласованный env-контракт. Для frontend в нем задаются публичный адрес витрины, адрес CMS, API-токен и preview-секрет. Для CMS задаются внешний адрес, проху-настройка, параметры rebuild-контура, параметры БД и секреты Strapi. Конкретные значения и учетные данные остаются ручной эксплуатационной настройкой и не экспортируются в репозиторий как версионируемые секреты.

Доказательная база контура развертывания разделена по уровням. Репозиторий подтверждает наличие Dockerfile, compose.yml и env-контракта; локальный контур проверки подтверждает успешные production build frontend и CMS, а также валидацию итоговой compose-конфигурации; внешний Dokploy-сегмент подтверждается отдельными платформенными артефактами и стендовой валидацией. Поэтому в тексте ВКР корректно говорить именно о Dokploy-ориентированном контуре развертывания с версионируемыми пакетами приложения в репозитории и внешним состоянием платформы на стороне оркестратора.

Платформенная привязка публичного frontend-домена показана на рисунке 2.7: для приложения astro зафиксированы служебный домен Dokploy и пользовательский домен с HTTPS, что соответствует роли Dokploy как внешней среды публикации витрины.

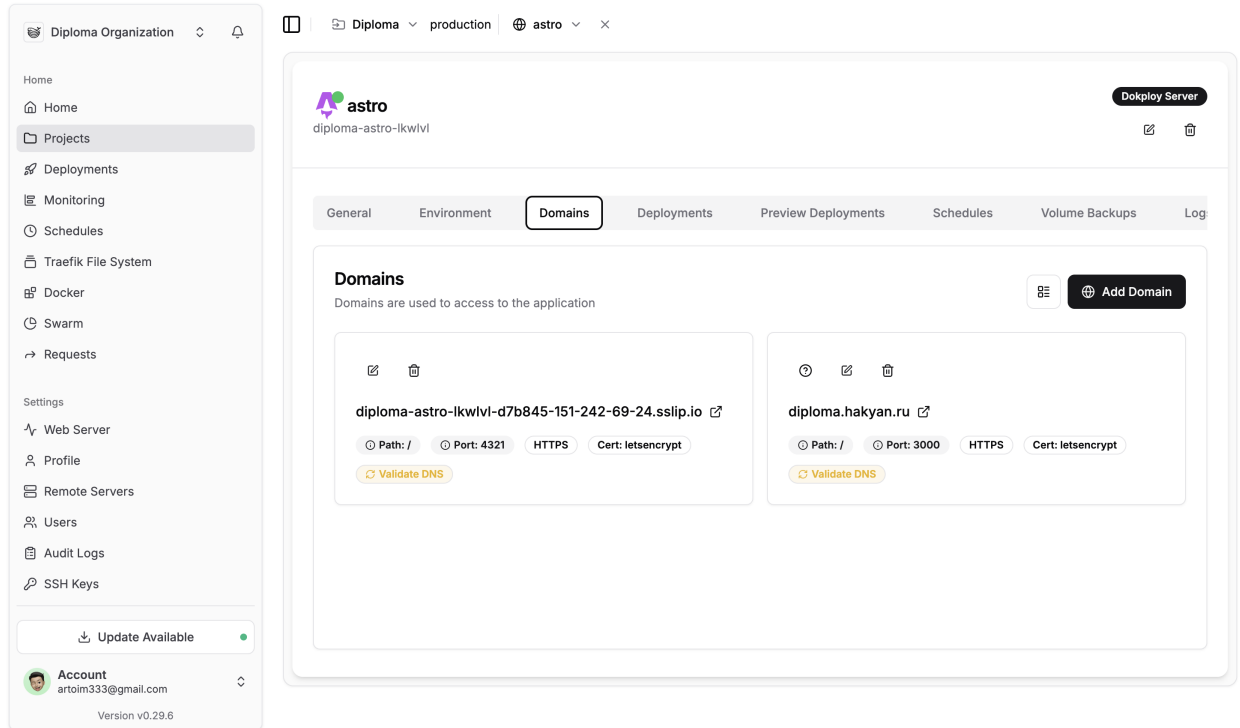


Рисунок 2.7 – Доменная привязка frontend-приложения в Dokploy: служебный домен, пользовательский домен и HTTPS

Такое разделение важно и методически. В тексте ВКР контур развертывания описывается не как устная инструкция по ручному запуску, а как формализованная схема, где пакеты приложений, инфраструктурные зависимости и публикационный механизм выражены в репозитории, а внешняя платформа выступает исполнителем этих артефактов.

Локально был подтвержден корректный синтаксис и связность compose-контура через валидацию итоговой конфигурации. Дополнительно production build самого приложения Strapi был выполнен успешно; отдельно воспроизводился и production build frontend с генерацией публичных маршрутов и sitemap. При этом сама конфигурация Dokploy как состояние внешней платформы относится к внешнему инфраструктурному сегменту. Поэтому данный результат корректно описывать как Dokploy-ориентированный контур развертывания, зафиксированный в репозитории, локально подтвержденный сборками приложений и конфигурационной валидацией, а также дополнительно подтвержденный внешней end-to-end валидацией на стенде проекта.

2.8 БЕЗОПАСНОСТЬ ПУБЛИЧНЫХ СЦЕНАРИЕВ

Валидация публичных форм

Публичные формы являются главным прикладным контуром безопасности витрины, поскольку именно здесь внешний пользователь взаимодействует с системой не только на чтение, но и на создание данных. Поэтому задача этого раздела состоит не в описании абстрактной защиты CMS, а в показе того, как система ограничивает публичную поверхность ввода и разводит маркетинговый и карьерный сценарии.

Для формы отклика на вакансию реализован двухступенчатый слой проверки. На клиенте используются `zod` и `react-hook-form`; пользователь обязан указать имя, email и телефон, подтвердить согласие на обработку персональных данных и прикрепить резюме допустимого типа `.pdf/.doc/.docx` размером не более 10 MB. Дополнительным антиспам-механизмом служит `honeypot`-поле. После клиентской проверки данные передаются не напрямую в CMS, а в серверный маршрут `POST /api/vacancy-applications`, который повторно валидирует `FormData` и только затем записывает `vacancy-application` через `CMS_API_TOKEN`. Тем самым браузер не получает прямой привилегированный доступ к `Strapi`, а отклик сохраняется в отдельной прикладной сущности.

Для маркетинговой формы `lead-form` применяется тот же принцип, но с другим прикладным результатом. Клиентская форма валидируется через `zod` и `react-hook-form`, требует подтверждения согласия, ограничивает длину полей и использует скрытое `honeypot`-поле. Отправка выполняется через `frontend`-маршрут `/api/lead-submissions`; внутри него `payload` проверяется повторно, после чего создается запись `lead-submission` с дополнительной атрибуцией по пути, заголовку страницы, локали и имени формы. Практически оба сценария подтверждены мутационным `smoke`, HTTP-проверками и появлением новых записей в `lead_submissions/vacancy_applications`. Их текущая граница охвата сформулирована явно: в системе уже есть серверная валидация, `consent` и `honeypot`, но не заявляются `rate limit`, CRM/email automation и расширенный сценарий обработки после отправки. Значение `CMS_API_TOKEN` при этом остается ручным эксплуатационным шагом, а не версионизируемым секретом репозитория.

Публичные справочники карьерного модуля

Публичный контур карьерного модуля требует публично доступных справочников для фильтрации вакансий, но не требует открывать дополнительные write-операции. Поэтому в CMS отдельно зафиксированы публичные справочники `industry` и `job-role`, а их роуты ограничены действиями `find` и `findOne`. Это позволяет `VacancyExplorer` загружать значения для фильтров и карточек вакансий, сохраняя минимальную публичную поверхность доступа.

Данный контур подтверждается конфигурацией маршрутов CMS, настройкой `public role` только на чтение и runtime-поведением карьерного каталога. Его граница также сформулирована явно: справочники являются служебными данными только для чтения прикладного публичного контура `/:locale/vacancies/*` и поддерживают единый двуязычный контур основной витрины.

Роли, права и разграничение доступа

Ролевая модель нужна системе для того, чтобы редакторский процесс, публикация и системное администрирование были разведены не декларативно, а в воспроизводимой матрице полномочий. В административной панели Strapi зафиксированы четыре человеческие роли: `administrator`, `marketer/content-manager`, `editor` и `hr`. `Administrator` сохраняет полный системный доступ, включая пользователей, роли, токены, `webhook`-настройки и конфигурацию CMS. `Marketer/content-manager` ведет маркетинговый контур и публикует `global`, `home-page`, `page`, `article`, `author` и `project`; `editor` готовит те же сущности как черновики, но не получает права публикации; `hr` изолирован в пределах карьерного контура и управляет `vacancy`, `industry`, `job-role` и `vacancy-application`. Такая матрица совпадает с редакторским процессом: маркетинговый и карьерный контуры разделены по полномочиям, а публикация отделена от подготовки материала.

Технически эта схема закреплена как версионизируемый источник истины в `security-model.ts` и синхронизируется во время `Strapi bootstrap`. Для content API дополнительно зафиксированы роли `public` и `authenticated`: `public` получает только `find/findOne` для опубликованного контента, а `authenticated` в конечном `scope` остается без `permissions`. Поверх этого работают ограничения на уровне маршрутов: контентные сущности не открывают write-операции через публичный API, тогда как `lead-submission` и `vacancy-application` разрешают только `create`. Человеческие роли дополняются техническими каналами доступа:

CMS_API_TOKEN используется только серверными маршрутами frontend для записи форм, а PREVIEW_SECRET — для чтения черновиков в /preview/*.

Доказательная база этого контура опирается на security-model.ts, код bootstrap, route-конфигурации CMS, описание редакторского процесса и приемочные проверки границы между preview и публичным режимом. Его текущая граница охвата сформулирована аккуратно: версионизируемыми и воспроизводимыми являются сама матрица ролей и permissions, тогда как фактические admin users, значения секретов и выпуск CMS_API_TOKEN остаются ручным эксплуатационным шагом. Это не ослабляет контур безопасности, поскольку ручной шаг относится к эксплуатационной настройке секретов, а не к проектированию модели доступа.

2.9 ТЕСТИРОВАНИЕ

Организация проверки и полученные результаты

Тестирование разработанной системы было организовано как воспроизводимый набор приемочных проверок, выстроенный от функциональных и нефункциональных требований, зафиксированных в таблицах 1.4 и 1.5. Поэтому проверка охватывала не абстрактную “работоспособность сайта вообще”, а именно те эксплуатационные свойства, которые были заявлены в первой главе: независимый редакторский цикл, основную витрину ru/en, управляемые через CMS SEO/Open Graph, защищенный preview, публичные формы и публикацию через схему webhook -> rebuild.

Контрольная локальная серия проверок была повторно зафиксирована 31 мая 2026 года. Для нее были использованы отдельные процессы Strapi, runtime frontend и preview-сервера для собранной витрины, а также workspace-команды pnpm smoke:front, pnpm evidence:testing, pnpm audit:browser, pnpm build:front и pnpm build:cms. Мутационная часть приемки не оформлялась как отдельная несвязанная проверка: она включалась для той же smoke-команды через переменную SMOKE_ALLOW_MUTATIONS=true и тем самым сохраняла общую логику приемочной проверки при явном разделении read-only и data-changing сценариев.

Такое разделение принципиально для корректной интерпретации результатов. Read-only smoke фиксирует маршрутные, preview- и build-инварианты без изменения данных. Mutation checks проверяют только те сценарии, в кото-

рых допустимо осознанное изменение прикладных записей, то есть отправку `lead-submission` и `vacancy-application`. `Browser audit` отвечает за браузерное выполнение репрезентативных страниц, а `build/sitemap-` и проверки по данным БД подтверждают соответственно состав публичного артефакта и факт прикладных или публикационных побочных эффектов. Отдельно от локального контура сохраняется короткая ручная демонстрационная приемка для защиты: она используется как показ уже подтвержденных сценариев, а не как источник более сильного статуса доказательства.

`Read-only smoke` подтвердил редирект `/ -> /ru/`, доступность маршрутов `/ru/` и `/en/`, работу репрезентативных CMS-страниц, статей, проектов и вакансий, корректность `legacy redirect`-правил, наличие `title`, `canonical`, `Open Graph`, `html[lang]` и `h1`, а также `preview`-поведение для `home-page`, `page`, `article`, `project` и `vacancy`. В этой же серии были воспроизведены отказ по неверному секрету, переход на опубликованный маршрут для статуса `published`, установка `httpOnly cookie` для статуса `draft` и `robots=noindex, nofollow` на `preview`-страницах. Результат `read-only` прогона составил `0 failures / 1 warning`; единственное предупреждение относилось к отключенным `mutation checks` и тем самым фиксировало не ошибку системы, а границу данного прогона.

Мутационная часть приемки показала, что `runtime`-сценарии публичных форм не ограничиваются только валидацией интерфейса. При включении `mutation checks` были подтверждены 400 для невалидных отправок, 201 для валидных `lead-submission` и `vacancy-application`, `same-origin guard` для формы отклика, файловые ограничения для резюме и появление новых записей в таблицах `lead_submissions` и `vacancy_applications`. Соответственно DB-подтверждение здесь интерпретируется как фиксация побочного эффекта уже проверенного пользовательского сценария, а не как самостоятельная замена `runtime`-приемки.

Сборочные и публикационные доказательные артефакты выделялись отдельно от браузерных и `runtime`-проверок. Команда `pnpm build:front` успешно сформировала 37 предсобранных HTML-маршрутов, `sitemap-index.xml` и `sitemap-0.xml`; анализ карты сайта подтвердил 31 публичный URL, включая `/ru/`, `/en/`, детальные маршруты с языковым префиксом для `articles/projects/vacancies`, при отсутствии `legacy URL` без локали. На стороне SQLite отдельно подтверждено наличие активного `Frontend rebuild hook` в `strapi_webhooks`, указывающего на CMS `/api/rebuild`. Это локально доказывает регистрацию автоматически создава-

емого webhook и маршрутной точки входа публикации, а не подменяет внешний платформенный rebuild недоказанным предположением.

Дополнительный браузерный слой был зафиксирован командой `pnpm audit:browser`. Проверка на четырех репрезентативных маршрутах завершилась с 0 failures и подтвердила наличие `html[lang]` и `h1`, доступность форм, отсутствие необозначенных элементов формы, ошибок страницы и `same-origin` сбоев выполнения и загрузки ресурсов, а также фиксацию базовых измерений `navigation timing`. Этот слой нужен не для подмены полного WCAG-аудита или production Web Vitals, а для подтверждения того, что приемочная проверка выдерживается не только на уровне HTML-ответов и build-артефактов, но и в реальном браузерном исполнении. Сводная трассировка ключевых контуров проверки представлена в таблице 2.1.

Таблица 2.1 – Трассировка требований, способов проверки и интерпретации результатов

Контур / требование	Способ проверки	Результат	Корректная интерпретация
Основная витрина <code>ru/en</code> и <code>locale-prefixed</code> маршрутный контур карьерного модуля	Read-only smoke, runtime-проверки маршрутов, build/sitemap-артефакты	Подтверждено: доступны <code>/ru/</code> , <code>/en/</code> , репрезентативные маршруты <code>page/article/project/vacancy</code> , а также редиректы со старых <code>/articles/*</code> , <code>/projects/*</code> и <code>/vacancies/*</code>	Доказывается работоспособность единой двуязычной витрины с <code>locale-prefixed</code> адресацией маркетинговых, редакционных и карьерных разделов.
Управляемые через CMS SEO/Open Graph, индексруемость и sitemap	Read-only smoke, build output, browser audit, проверка sitemap	Подтверждено: присутствуют <code>title</code> , <code>canonical</code> , <code>og:*</code> , корректная публичная индексруемость и файлы <code>sitemap-index.xml/sitemap-0.xml</code>	Доказывается единый контур метаданных для <code>home-page</code> , <code>page</code> и детальных страниц сущностей при метаданных списковых страниц, формируемых на уровне маршрутов; этот результат не превращается в утверждение о полном редакторском управлении SEO для всех списковых разделов.
Защищенный <code>preview</code> и разведение <code>draft/public</code> состояний	Read-only smoke, runtime-проверки <code>preview</code> , репрезентативная браузерная проверка сценария	Подтверждено: по неверному секрету возвращается 401, выполняется 307 redirect, устанавливается <code>draft-cookie</code> и выставляется <code>noindex</code> для <code>home-page</code> , <code>page</code> , <code>article</code> , <code>project</code> , <code>vacancy</code>	Доказывается управляемая граница между опубликованным и черновым контентом; результат относится к серверному <code>preview</code> -контур и не трактуется как отдельная пользовательская система аутентификации.
Публичные формы и прикладные записи	Mutation checks, HTTP-проверки валидации, данные SQLite	Подтверждено: воспроизводятся ответы 400/201, защита <code>same-origin</code> , ограничения на файлы и рост <code>lead_submissions</code> и <code>vacancy_applications</code>	Доказывается серверная валидация и сохранение результатов валидной отправки; это подтверждение прикладного сценария, а не нагрузочная или эксплуатационная метрика очередей, почты или внешних интеграций.

Продолжение таблицы 2.1

Контур / требование	Способ проверки	Результат	Корректная интерпретация
Сборка публичной витрины и sitemap	pnpm build:front, проверка dist/client, pnpm evidence:testing	Подтверждено: зафиксированы 37 предсобранных HTML-маршрутов и 31 URL в sitemap-0.xml	Доказывается состав артефакта сборки, готового к публикации, и покрытие публичных маршрутов; размер dist/client и число маршрутов интерпретируются как свойства контрольной сборки, а не как прямые пользовательские метрики производительности.
Публикационный путь webhook -> rebuild	Данные SQLite для strapi_webhooks, анализ кода и сборки, внешняя стендовая валидация от 1 июня 2026 года	Подтверждено: активен Frontend rebuild hook, локально доступен CMS /api/rebuild, на стенде воспроизведен полный путь до Dokploy rebuild/redeploy	Локальные данные подтверждают версионизируемую точку входа публикации и регистрацию webhook; внешняя стендовая валидация отдельно подтверждает платформенный путь выполнения без подмены локального факта состоянием внешней инфраструктуры.
Браузерная доступность и выполнение страниц во время работы	pnpm audit:browser на репрезентативных маршрутах	Подтверждено: 4 маршрута, 0 failures	Доказывается локальный браузерный контур проверки структуры документа, форм и ошибок исполнения; этот результат не следует называть полным WCAG-аудитом или проверкой production-Web Vitals.
Демонстрационная приемка для защиты	Короткий ручной показ репрезентативных маршрутов и preview-сценария	Репрезентативная демонстрация	Сценарий служит воспроизводимым подтверждением уже проверенных путей и не повышает статус доказательства сверх автоматизированных, сборочных и базовых проверок данных.

Количественные показатели контрольной серии проверок

Наряду с качественной трассировкой были зафиксированы и количественные результаты контрольной серии. В таблицу 2.2 включены только те значения, которые непосредственно измерялись в локальной контрольной серии; архитектурные выводы или границы score не подаются здесь как “метрики”.

Таблица 2.2 – Ключевые показатели контрольной серии проверок

Показатель	Значение	Граница интерпретации
Read-only smoke	0 failures / 1 warning	Подтверждены обязательные маршрутные инварианты, требования к SEO и preview; предупреждение относится только к невключенной мутационной части проверки
Mutation checks	0 failures / 0 warnings	Подтверждены сценарии изменения данных для lead-submission и vacancy-application; результат относится к прикладным отправкам, а не к полной нагрузочной модели форм
Browser audit	4 маршрута / 0 failures	Зафиксирован браузерный контур для репрезентативных страниц; численное значение относится к текущей выборке маршрутов, а не ко всему множеству страниц витрины

Продолжение таблицы 2.2

Показатель	Значение	Граница интерпретации
Сборка frontend	37 предсобранных HTML-маршрутов	Зафиксирован объем публичного результата сборки; число маршрутов характеризует результат контрольной серии от 31 мая 2026 года
Sitemap coverage	31 URL в sitemap-0.xml	Подтвержден состав публичной карты сайта для основной витрины, локализованных контентных маршрутов и карьерного раздела в его текущей маршрутной границе
Объем dist/client	около 2.4 MB	Зафиксирован размер build-артефакта публичной витрины; он описывает состояние контрольной сборки и не подменяет полноценное браузерное измерение производительности

Практическое значение результатов

Короткая ручная приемка в этом контуре имеет отдельную роль. Она воспроизводит редирект на /ru/, открытие русской и английской версии основной витрины, показ репрезентативной CMS-страницы, запуск preview из Strapi, открытие детальной вакансии и проверку невалидного ввода для маркетинговой и карьерной формы. Тем самым на защите демонстрируются именно те маршруты и сценарии, которые уже подтверждены автоматизированными, браузерными, сборочными и базовыми проверками данных.

Практическое значение результатов состоит в том, что ключевые требования первой главы доведены до состояния воспроизводимого набора приемочных проверок. Независимый редакторский цикл подтверждается сочетанием CMS-страницы, Dynamic Zone и защищенного preview; основная витрина ru/en — маршрутными и сборочными проверками; управляемые через CMS SEO/Open Graph — проверками метаданных и sitemap; защищенные публичные формы — мутационной приемкой и прикладными записями в БД; публикация через webhook -> rebuild — локальной регистрацией автоматически регистрируемого webhook и отдельной внешней стендовой валидацией полного пути publish/unpublish -> rebuild/redeploy. В результате зрелость решения подтверждается не общим тезисом о “наличии функций”, а воспроизводимым соответствием между требованием, способом проверки и наблюдаемым результатом.

Корректная интерпретация этой доказательной базы заключается в следующем: для заявленного дипломного score система уже подтверждена как рабочий инженерный контур управления маркетинговым контентом, в котором изменение страницы, предварительная проверка, публикация и прикладная обработка

пользовательских отправок не зависят от неформальных ручных действий в коде. При этом каждый класс доказательств сохраняет свою границу: `browser audit` характеризует локальный браузерный контур, размер `dist/client` описывает артефакт сборки, а внешний `Dokploy` путь фиксируется как отдельный стендовый факт. Такая интерпретация делает выводы по `testing section` строгими: они показывают зрелость решения в границах измеренного приемочного контура и не приписывают системе свойства, которые в данной работе специально не измерялись.

2.10 ВЫВОДЫ ПО ПРОЕКТНОЙ ГЛАВЕ

Реализация проекта образует рабочий CMS-first контур, в котором `Strapi` выступает ядром управления контентом, а `Astro` — слоем публикации и рендеринга. В репозитории зафиксированы самостоятельные приложения CMS и витрины, предметно ориентированная модель данных для статей, проектов, вакансий, откликов, маркетинговых лидов, глобальных данных и маркетинговых страниц, а также единый `OpenAPI`-контур интеграции между `backend` и `frontend`.

Практически значимым результатом является перевод главной витрины и `pages`-маршрутов в управляемую CMS-модель. Главная страница и маркетинговые `slug`-страницы собираются из `Dynamic Zone`, используют локализованные маршруты `ru/en`, получают `SEO/Open Graph` метаданные из CMS и поддерживают защищенный `preview`-сценарий. Дополнительно в системе функционируют публичные разделы статей, проектов и вакансий; их детальные страницы используют тот же `SEO`-контур и тот же механизм `preview`-метаданных. Карьерный модуль при этом включает прикладной пользовательский путь от просмотра вакансии до отправки отклика. Вместе с этим `page builder` уже поддерживает собственный блок маркетинговой формы и отдельный сценарий сбора лидов через серверный маршрут `frontend`.

Дополнительным результатом стало оформление публикационного контура и контура развертывания как отдельного инженерного слоя. В репозитории зафиксированы `webhook`-связка `publish/unpublish -> rebuild`, внешний контракт повторной сборки `frontend` и `Docker`-контур CMS. За счет этого вторая глава описывает не только CMS-модель и маршруты витрины, но и воспроизводимый эксплуатационный процесс публикации и развертывания.

Таким образом, вторая глава описывает не абстрактный замысел, а реальную архитектуру и реализованные механизмы управления маркетинговым контентом: разделение CMS и витрины, предметную модель данных, page builder на основе Dynamic Zone, локализованную витрину первой очереди, управляемый мета-контур, публикационный контур и контур развертывания, а также безопасный доступ к черновому контенту через preview.

ЗАКЛЮЧЕНИЕ

Итоги решения поставленных задач

Цель работы, заключающаяся в разработке системы управления контентом для корпоративного маркетингового сайта на базе Strapi и Astro, достигнута в объеме, зафиксированном для данной ВКР. В ходе исследования была проанализирована предметная область маркетинговых сайтов, показаны ограничения подхода, при котором контент и публичная витрина остаются жестко связаны, и обоснован выбор CMS-first архитектуры с разделением контентного и frontend-слоев. На этой основе были сформированы функциональные и нефункциональные требования к системе, а также спроектированы ее архитектура, модель данных и редакторские сценарии.

В практической части работы реализован контур управления ключевыми сущностями через Strapi, включая home-page, page, article, project, vacancy, связанные справочники и прикладные сущности форм. Для маркетинговых страниц разработан конструктор на основе Dynamic Zone, а витрина на Astro реализована по CMS-first модели с локализованными маршрутами, режимом предварительного просмотра, SEO/Open Graph, sitemap и публикационной связкой webhook -> rebuild. Тем самым поставленные задачи по анализу, проектированию, реализации и проверке решения были последовательно решены и связаны в единый инженерный результат.

Практический результат работы

Практическим результатом ВКР стала рабочая CMS-first платформа, в которой Strapi выступает ядром управления контентом, а Astro — публичной витриной корпоративного маркетингового сайта. Контент-менеджер или маркетолог получает возможность управлять главной страницей, маркетинговыми slug-страницами, статьями, проектами, SEO-метаданными и глобальными данными сайта через административный интерфейс, не изменяя исходный код frontend-приложения. Для карьерного контура реализованы отдельные сущности вакансий, справочников и откликов, а прикладные формы вынесены в серверные маршруты frontend с повторной валидацией и защитой публичного ввода. Тем самым маркетинговая двуязычная витрина и кадровый публичный сценарий сведены в одну систему и публикуются в общем locale-prefixed route-контуре.

Эксплуатационный контур при этом оформлен как самостоятельная часть архитектуры, а не как вспомогательная операция после разработки. Финальная deployment-модель строится вокруг Dokploy: frontend и CMS определены как два отдельных Docker-приложения, для CMS выделен отдельный контракт доступа к PostgreSQL, а публикация в Strapi управляет обновлением витрины через автоматически регистрируемый webhook и нативный rebuild hook frontend-приложения. Тем самым публикация и deployment сведены в единый управляемый инженерный контур, локально подтвержденный на стороне приложений, конфигурации и webhook-связки.

Существенным достоинством полученного результата является то, что он подтвержден не только структурой кода, но и воспроизводимой доказательной базой. В контрольной серии проверок зафиксированы успешная сборка frontend и CMS, генерация 37 предсобранных публичных маршрутов, наличие sitemap, корректная работа preview-сценариев, SEO-контур, локализованных маршрутов ru/en, публичных форм и webhook-связки публикации, включая регистрацию автоматически создаваемого webhook пересборки в strapi_webhooks. Автоматизированная smoke-проверка подтвердила обязательные сценарии без отказов, а браузерный аудит показал стабильное выполнение репрезентативных маршрутов без page errors и same-origin ошибок выполнения и загрузки ресурсов. Это позволяет рассматривать результат работы как завершенное прикладное решение для рассматриваемой предметной области, а не как набор разрозненных технических экспериментов.

Ограничения решения

Несмотря на достигнутую целостность решения, итоговая версия системы осознанно ограничена рамками ВКР. Публичный двуязычный контур охватывает основную витрину, маркетинговые страницы, статьи, проекты и вакансии. Локализуемая CMS-модель для vacancy, industry и job-role, а также локализованный preview и production-маршруты `/:locale/vacancies/*` входят в реализованный результат. Для разделов-списков articles, projects и vacancies используется слой метаданных, формируемый на уровне маршрутов, тогда как SEO, управляемое из CMS, в обязательном объеме доведено до home-page, page и детальных страниц сущностей article/project/vacancy.

Дополнительно следует учитывать, что публикационный контур доказан локально как зафиксированный в репозитории, прошедший конфигурационную

валидацию, подтвержденный сборками приложений и связанный с управляемым webhook. Дополнительно 1 июня 2026 года на внешнем стенде проекта был подтвержден и полный путь Strapi -> webhook -> Dokploy rebuild/redeploy -> обновленная Astro-витрина. Тем самым контур развертывания и публикации доказан не только на локальном воспроизводимом наборе проверок, но и отдельной внешней end-to-end валидацией. При этом тестовый контур работы в целом остается приемочным и интеграционным набором проверок, а не полным набором unit- и end-to-end тестов для всех возможных ветвей поведения. Эти ограничения не обесценивают результат ВКР, но задают корректную границу его доказанности и предотвращают завышение фактически подтвержденных свойств системы.

Направления дальнейшего развития

Логичным продолжением работы является расширение набора Dynamic Zone-блоков, углубление редакторских процессов и развитие эксплуатационной автоматизации. В прикладном плане это может включать расширение мультиязычного публичного контура карьерного модуля, если такая задача станет отдельным продуктовым этапом, дальнейшее развитие ролей и редакторских сценариев, а также подключение внешних интеграций для обработки лидов и откликов. Отдельным направлением является развитие эксплуатационной наблюдаемости контура развертывания, включая платформенные артефакты Dokploy, журналирование событий публикации и дополнительные эксплуатационные метрики.

Таким образом, выполненная работа создает завершенную архитектурную и программную основу для дальнейшего развития корпоративной маркетинговой платформы. Главный итог ВКР состоит в том, что управление контентом, публикация, локализация, preview, SEO-контур и базовая эксплуатационная модель были сведены в единую CMS-first систему, пригодную для практического использования и дальнейшего масштабирования.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

- API** Application Programming Interface — интерфейс программирования приложений.
- CI/CD** Continuous Integration / Continuous Delivery — непрерывная интеграция и доставка.
- CMS** Content Management System — система управления контентом.
- CRUD** Create, Read, Update, Delete — базовые операции управления данными.
- DTO** Data Transfer Object — объект передачи данных между слоями приложения.
- HR** Human Resources — кадровая функция и связанные с ней бизнес-процессы.
- HTML** HyperText Markup Language — язык разметки гипертекста.
- HTTP** HyperText Transfer Protocol — протокол передачи гипертекста.
- REST** Representational State Transfer — архитектурный подход к построению веб-API.
- SEO** Search Engine Optimization — поисковая оптимизация.
- UI** User Interface — пользовательский интерфейс.
- URL** Uniform Resource Locator — унифицированный адрес ресурса.
- WCAG** Web Content Accessibility Guidelines — рекомендации по обеспечению доступности веб-контента.
- WCMS** Web Content Management System — веб-система управления контентом.

СЛОВАРЬ ТЕРМИНОВ

CMS-first архитектура Подход, при котором CMS выступает центром управления контентом, а публичная витрина реализуется как отдельное приложение.

Dynamic Zone Механизм Strapi, позволяющий собирать страницу из переиспользуемых блоков разного типа.

Headless CMS Система управления контентом, отделяющая хранение и редактирование данных от их публичного отображения.

Контент-модель Набор сущностей, полей и связей, определяющий структуру данных в CMS.

Локализованный маршрут Публичный URL, зависящий от выбранной локали, например /ru/... или /en/....

Пайплайн сборки Последовательность автоматических шагов проверки, сборки и публикации проекта.

Публикационный контур Согласованная цепочка действий и компонентов, через которую опубликованный в CMS контент попадает на публичную витрину.

Публичная витрина Frontend-часть системы, которая получает данные из CMS и публикует их в виде пользовательских веб-страниц.

Режим предварительного просмотра Механизм безопасного просмотра черновой версии материала до его публикации.

Роутинг Механизм сопоставления URL-адресов с обработчиками или страницами приложения.

Серверный рендеринг Формирование HTML на сервере перед отправкой результата клиенту.

Slug Человекочитаемый идентификатор страницы или записи, используемый в составе URL-адреса.

Типизация контента Формальное описание структуры данных для валидации и безопасного обмена.

Webhook HTTP-вызов, автоматически отправляемый одной системой другой при наступлении определенного события.

СПИСОК ИЛЛЮСТРАТИВНОГО МАТЕРИАЛА

Список иллюстраций

2.1	Редактирование CMS-страницы в Strapi: slug, Dynamic Zone, SEO и доступ к preview	30
2.2	Публичная витрина в маршруте с языковым префиксом /en/, опубликованная на основе данных из CMS	31
2.3	Детальная страница вакансии с подключенной формой отклика как прикладной публичный сценарий	33
2.4	Предпросмотр черновой версии CMS-страницы в интерфейсе Strapi	36
2.5	Автоматически регистрируемый webhook пересборки в Strapi: целевой URL и подписка на entry.publish/entry.unpublish	40
2.6	Рабочее окружение в Dokploy с отдельными сервисами astro, strapi и pg	41
2.7	Доменная привязка frontend-приложения в Dokploy: служебный домен, пользовательский домен и HTTPS	42
2.8	Контекстная схема CMS-first системы на базе Strapi и Astro . .	63
2.9	Модель данных CMS с основными сущностями, связями и общими атрибутами публикации	64
2.10	Жизненный цикл локализуемой контентной единицы в контуре draft/publish	65
2.11	Сценарий защищенного предварительного просмотра чернового контента	66

Список таблиц

1.1	Связь особенностей маркетингового сайта с инженерными требованиями к системе	10
1.2	Сравнение традиционной CMS и headless CMS по критериям маркетингового сайта	14
1.3	Критерии выбора стека для CMS-first маркетинговой платформы	16
1.4	Трассировка функциональных требований к системе	19
1.5	Трассировка нефункциональных требований к системе	21

2.1	Трассировка требований, способов проверки и интерпретации результатов	47
2.2	Ключевые показатели контрольной серии проверок	48

СПИСОК ЛИТЕРАТУРЫ

1. *Торгунакова Е. В., Торгунаков Е. А.* Веб-сайт организации как инструмент реализации комплекса маркетинговых коммуникаций // Экономика и управление. — 2023. — Т. 29, № 6. — С. 645—652. — DOI: 10.35854/1998-1627-2023-6-645-652. — URL: <https://cyberleninka.ru/article/n/veb-sayt-organizatsii-kak-instrument-realizatsii-kompleksa-marketingovyh-kommunikatsiy> (дата обращения 24.05.2026).
2. *Воронюк М. П.* Изучение потребностей и анализ задач для системы управления контентом в организациях розничной торговли : Тезисы доклада / Университет ИТМО. — 2020. — URL: <https://kmu.itmo.ru/file/download/application/11149> (дата обращения 24.05.2026).
3. *Беляев А. А.* Навигация как ключевой компонент визуальной организации веб-сайта // Медиаскоп. — 2009. — № 2. — URL: <https://cyberleninka.ru/article/n/navigatsiya-kak-klyuchevoy-komponent-vizualnoy-organizatsii-veb-sayta> (дата обращения 24.05.2026).
4. *Михайлова Н. А.* Разработка системы навигации для управления информационным контентом консалтинговой компании : Bachelor's thesis / Михайлова Наталия Анатольевна. — Санкт-Петербургский политехнический университет Петра Великого, 2016. — URL: <https://elib.spbstu.ru/dl/2/v16-1111.pdf/info> (дата обращения 24.05.2026).
5. *Машина Е. А.* Использование централизованной системы управления контентом в качестве основы корпоративного решения управления знаниями : Тезисы доклада / Университет ИТМО. — 2022. — URL: <https://kmu.itmo.ru/file/download/application/24737> (дата обращения 24.05.2026).
6. *Есиков А. В.* Сравнительный анализ систем управления контентом (content management system – CMS) // Перспективы развития информационных технологий. — 2013. — № 13. — URL: <https://cyberleninka.ru/article/n/sravnitelnyy-analiz-sistem-upravleniya-kontentom-sontent-management-system-cms> (дата обращения 24.05.2026).

7. *Bandirmali N.* mtCMF: A novel memory table based content management framework for automatic website generation // *Computer Standards & Interfaces*. — 2018. — Т. 58. — С. 43—52. — DOI: 10.1016/j.csi.2017.12.002.
8. Applying MDD in the content management system domain / D. Priefer [и др.] // *Software and Systems Modeling*. — 2021. — Т. 20, № 6. — С. 1919—1943. — DOI: 10.1007/s10270-021-00872-3. — URL: <https://link.springer.com/article/10.1007/s10270-021-00872-3> (дата обращения 24.05.2026).
9. *Gkantouna V., Tzimas G.* CMS-oriented Modeling Languages: An Attempt to Assist Model-driven Development in CMS Domain // *Proceedings of the 15th International Conference on Web Information Systems and Technologies*. — SCITEPRESS - Science, Technology Publications, 2019. — С. 372—379. — DOI: 10.5220/0008363103720379. — URL: <https://www.scitepress.org/PublishedPapers/2019/83631/83631.pdf> (дата обращения 24.05.2026).
10. *Filipe P., Ribeiro A., Rodrigues da Silva A.* XIS-CMS: Towards a Model-Driven Approach for Developing Platform-Independent CMS-Specific Modules // *Proceedings of the 4th International Conference on Model-Driven Engineering and Software Development*. — SCITEPRESS - Science, Technology Publications, 2016. — С. 535—543. — DOI: 10.5220/0005745805350543. — URL: <https://www.scitepress.org/Papers/2016/57458/57458.pdf> (дата обращения 24.05.2026).
11. *Santahuhta P.* Evaluating Headless CMSs Role in Web Development Utilizing the Analytic Hierarchy Process : Bachelor's thesis / Santahuhta Pyry. — LUT University, 2022. — URL: <https://lutpub.lut.fi/handle/10024/163897> (дата обращения 23.05.2026).
12. *Matjuhhin S.* The Development of an API-Driven Content Management System : Bachelor's thesis / Matjuhhin Saveli. — Tampere University of Applied Sciences, 2026. — URL: <https://www.theseus.fi/handle/10024/916906> (дата обращения 23.05.2026).
13. *Holčík F.* Headless CMS in Cloud : Bachelor's thesis / Holčík Filip. — Brno University of Technology, 2025. — URL: https://www.vut.cz/en/students/final-thesis/detail/166024?zp_id=166024 (дата обращения 23.05.2026).

14. *O'Leary D. E.* A multilingual knowledge management system: A case study of FAO and WAICENT // *Decision Support Systems*. — 2008. — Т. 45, № 3. — С. 641—661. — DOI: 10.1016/j.dss.2007.07.007.
15. Challenges for the multilingual Web of Data / J. Gracia [и др.] // *Journal of Web Semantics*. — 2012. — Т. 11. — С. 63—71. — DOI: 10.1016/j.websem.2011.09.001.
16. *Соловьев Э. Е.* Разработка мультязычного новостного Web-приложения : Bachelor's thesis / Соловьев Эдуард Евгеньевич. — Национальный исследовательский Томский политехнический университет, 2023. — URL: <https://earchive.tpu.ru/jspui/handle/11683/76463> (дата обращения 23.05.2026).
17. Welcome to the Strapi CMS Documentation! | Strapi 5 Documentation. — URL: <https://docs.strapi.io/cms/intro> (дата обращения 18.05.2026).
18. On-demand rendering | Astro Docs. — URL: <https://docs.astro.build/en/guides/on-demand-rendering/> (дата обращения 18.05.2026).
19. Strapi & Astro | Astro Docs. — URL: <https://docs.astro.build/ru/guides/cms/strapi/> (дата обращения 18.05.2026).
20. Monorepos | Nx. — URL: <https://nx.dev/docs/concepts/decisions/why-monorepos> (дата обращения 18.05.2026).
21. Web Content Accessibility Guidelines (WCAG) 2.2. — 2023. — URL: <https://www.w3.org/TR/WCAG22/> (дата обращения 23.05.2026).
22. Internationalization (i18n) Routing | Astro Docs. — URL: <https://docs.astro.build/en/guides/internationalization/> (дата обращения 23.05.2026).
23. Overview of International and Multilingual Site Topics | Google Search Central. — URL: <https://developers.google.com/search/docs/specialty/international> (дата обращения 23.05.2026).
24. *Cigoj P., Blazic B. J.* An Intelligent and Automated WCMS Vulnerability-Discovery Tool: The Current State of the Web // *IEEE Access*. — 2019. — Т. 7. — С. 175466—175473. — DOI: 10.1109/ACCESS.2019.2957573.

25. Горло Н. Е., Пескова О. Ю. Подходы к построению защищенной системы управления контентом // Известия Южного федерального университета. Технические науки. — 2010. — Т. 112, № 11. — С. 75—82. — URL: <https://cyberleninka.ru/article/n/podhody-k-postroeniyu-zaschischennoy-sistemy-upravleniya-kontentom> (дата обращения 24.05.2026).

The diagram illustrates the architecture of a Monorepo Nx + npm system, showing the flow of data and components between different parts of the application.

Monorepo Nx + npm

apps/cms Strapi 5

- Content types: page, home-page, global, article, author, project, vacancy, vacancy-application, lead-submission, industry, job-role
- Preview config: config/admin.ts
- REST API + OpenAPI docs
- Strapi Admin UI

apps/front Astro 6

- Preview routes: /api/preview, /preview/...
- Localized public routes: /ru/, /en/, /locale/:slug/
- Content routes: /locale/articles/, /locale/projects/, /locale/vacancies/, /locale/vacancies/:slug/

External Components and Data Flow:

- SQLite**: Used for storing data (хранение записей) and reading/writing data (чтение и запись данных).
- OpenAPI JSON**: Generated client articles/projects/vacancies, published API requests.
- Generated SDK + custom data layer**: Draft requests, draft fetchers, home-page + pages, articles/projects + vacancies.
- Layout + SEO rendering main.astro**: Articles, projects, and vacancies use locale-prefixed public routes: legacy/articles/, projects, and /vacancies redirect to the default locale.

Interactions:

- Strapi Admin UI** (Strapi 5) interacts with **Preview config** (preview action) and **REST API + OpenAPI docs** (ограничивает публичные GET status=published, формирует preview URL).
- REST API + OpenAPI docs** interacts with **SQLite** (хранение записей) and **OpenAPI JSON** (генерирует клиент articles/projects/vacancies, публикует схему API).
- Preview routes** (Astro 6) interacts with **REST API + OpenAPI docs** (draft requests, draft fetchers, home-page + pages) and **Localized public routes** (articles/projects + vacancies).
- Localized public routes** interacts with **Content routes** (articles, projects, and vacancies use locale-prefixed public routes).
- Content routes** interacts with **Layout + SEO rendering main.astro**.

Users:

- Контент-менеджер** (Content Manager) interacts with **Strapi Admin UI** and **Preview routes**.
- Посетитель сайта** (Website Visitor) interacts with **Localized public routes** and **Content routes**.

Рисунок 2.8 – Контекстная схема CMS-first системы на базе Strapi и Astro

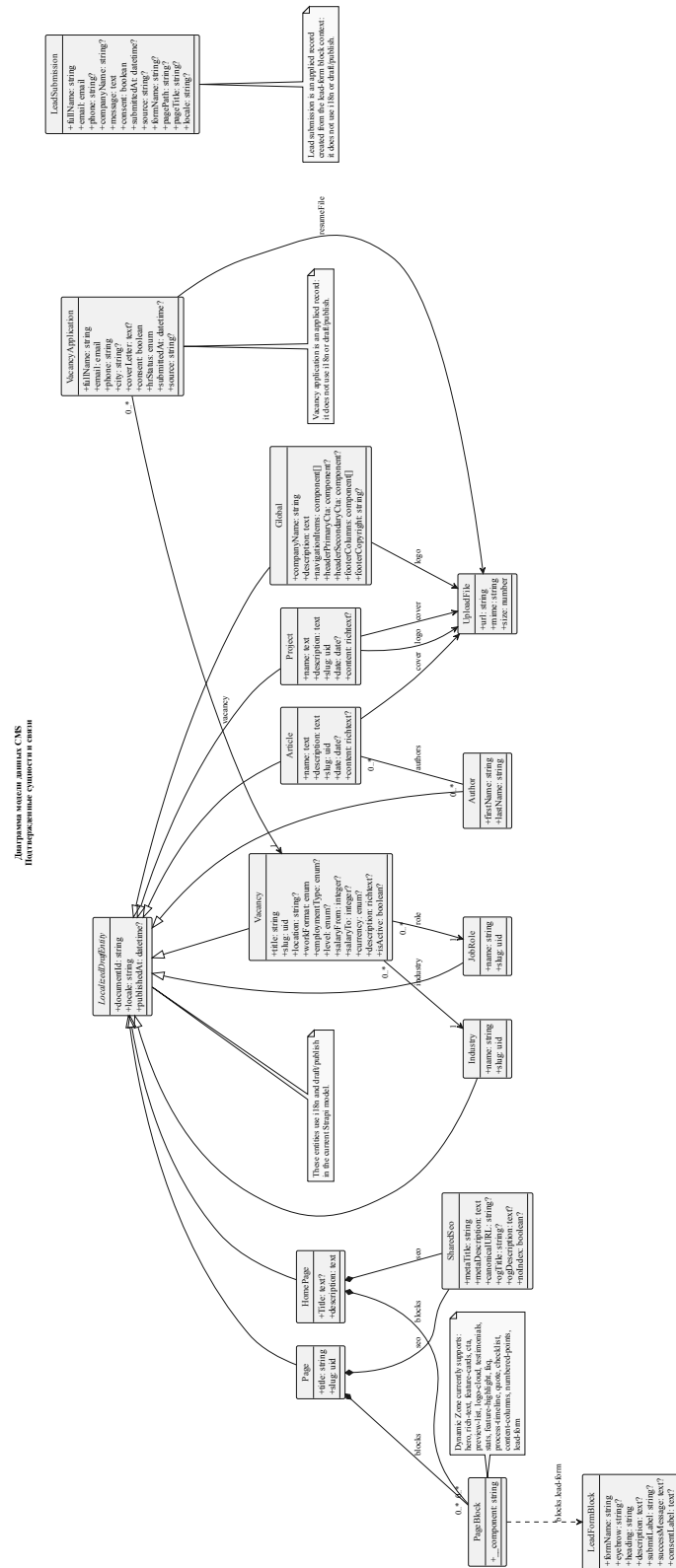


Рисунок 2.9 – Модель данных CMS с основными сущностями, связями и общими атрибутами публикации

Приложение В. Схема жизненного цикла контента

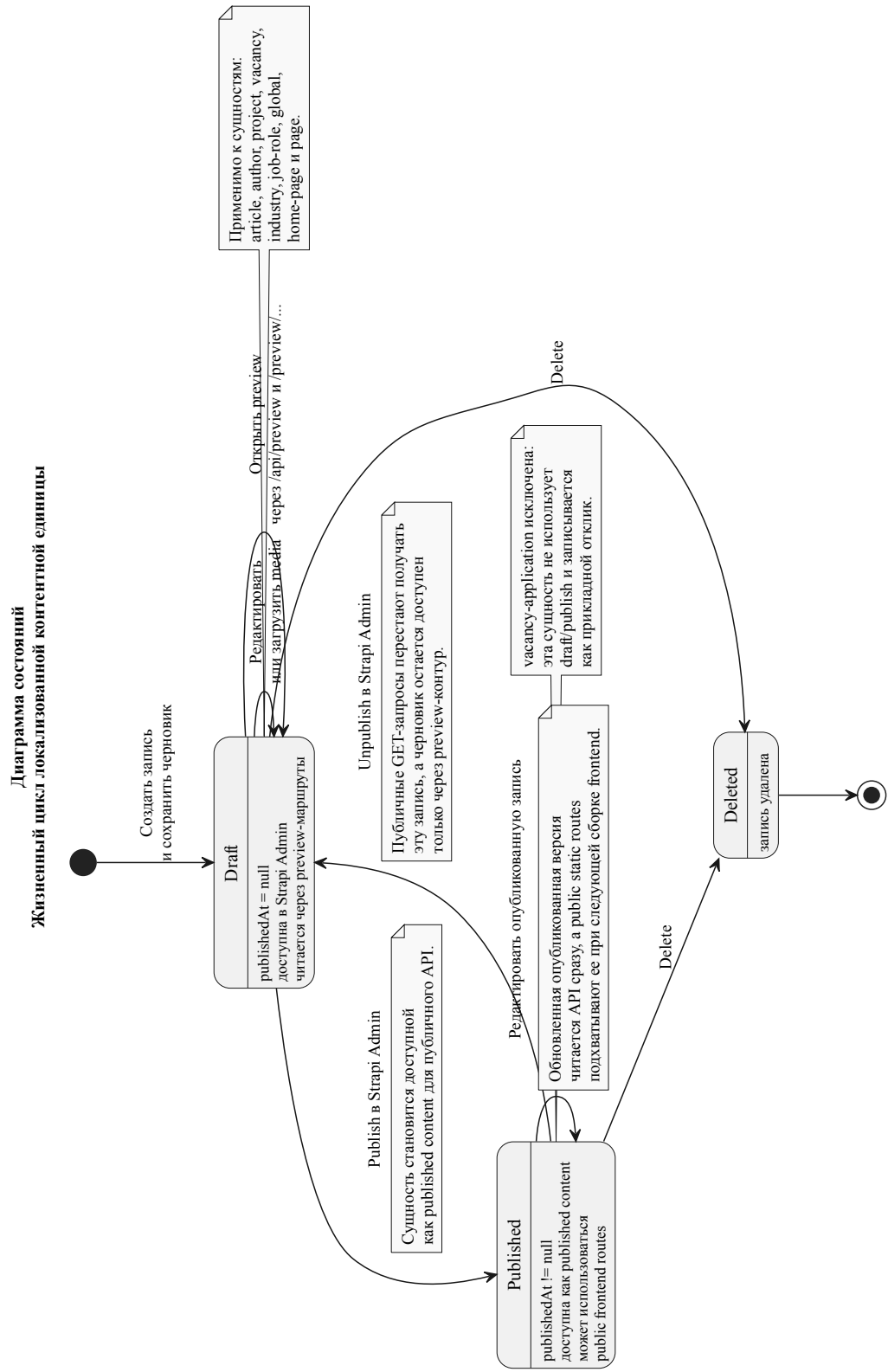


Рисунок 2.10 – Жизненный цикл локализуемой контентной единицы в контуре draft/publish



Рисунок 2.11 – Сценарий защищенного предварительного просмотра черного контента